Faculty of Sciences Department of Computer Sciences

MASTER THESIS

Fault-Tolerant Microarchitectures: Enhancing Reliability Through Redundant Execution

Author : DEVROYE Louis

Advisor : MÜHLBERG Jan Tobias

Director : MÜHLBERG Jan Tobias

Academic year 2025–2026

Abstract

Trusted execution environments promise isolation but say little about what happens after a crash. This thesis asks whether an enclave can rewind itself to safety without ever trusting the machine around, all of that while being cost-effective. The RISC-V architecture, through Keystone, becomes the testing ground, its physical memory protection and security monitor treated as the only fixed point in a hostile world. The host keeps the checkpoint, yet he never learns what it contains. Sealing keys bound to enclave measurement turn a blind, opaque blob into something the enclave alone can read and trust again. Confidentiality comes from AES counter mode, authenticity from a separate MAC key, and both survive a channel assumed to be actively adversarial. Rewind here means destroying a faulted enclave outright and rebuilding it fresh from the same binary, not nudging a wounded process back to life. Every save, every load, every unseal costs cycles, those cycles are measured in order to find the point where rewinding becomes efficient. Break-even thresholds emerge from this cost, shifting with fault frequency and checkpoint interval in ways that resist a single universal answer. Corrupted blobs, truncated writes, and single flipped bits are tested against the sealed checkpoint with the goal to make sure that the assumptions made are fulfilled with our implementation. Rollback prevention is explicitly left outside this boundary, a limitation on the host's untrustworthiness directly from the Keystone design. The results show that rewind is not to be considered an obvious choice, instead, the outcomes reveal a map of when it does become relevant. Long, repetitive, interruption-prone workloads justify the overhead whereas short or rare-fault workloads do not. The trade-off is clear: liveliness is bought at the price of sealing, unsealing, and a Security Monitor willing to rebuild trust from nothing.

Contents

Chapter 1

Introduction

Microprocessors are getting more efficient each year due to miniaturization of transistors [1]. However, this rise in efficiency comes with the pricey cost that are faults. Critical systems are particularly vulnerable considering the stakes (human lives) and the real time constraints such as time, temperature, computational power or even voltage. This field is not new [2, 3] but is still drawing attention, has a lot of gaps yet to be covered and is structuring, mostly around the type of faults and the way to deal with them [4].

Fault detection is a wildly studied field [5], it is a pillar for critical systems and for networks. The detection goes along with Error Correction Codes (ECC), these are mostly used in network due to the necessity to get the good informations using as little time/computation as possible. However, these techniques come with a lot of overhead and assumptions that cannot be made in this field. First, network considers the source to be viable, when a request is made to a server, the informations are checked, let's say with a basic checksum. In micro architectures, the sensors would be the parallel to the server. However, the data given by the sensor are not known to be true or false (exception made for specific cases, for example if a distance check returns a negative answer). Secondly, all the layers needed to prevent a fault from happening, in TCP for example, would be too costly to just put in a micro processor, in overhead (from both computation and memory standpoint) and in time needed. In fact, the main topic is that these systems are time bounded, so it is important to be able to have the, correct, data before a certain deadline. Unfortunately, all these constraints do not apply, at least not all at once, to networks.

On the other hand, a lot of programs and studies are focused on *Software Implemented Fault Tolerance* (SIFT) and more precisely on *time redundancy* methods, they are solely interested in making fault-tolerant programs that meet deadlines. That is in itself an honorable (and complex) goal, yet one of the main problem, related to this topic, with miniaturization being the increase in temperature. One could argue that making extra computation or overclock the CPU when an error occurs, possibly because of the aforementioned heat, wouldn't make any sense. In fact, it could slow the whole system (to cool down) or cause more faults at the time being and it would strain the system even more in the long run [6, 7].

To improve this contradictory aspect, some research has been done. They put the light on other aspects than only timing constraints (while keeping the reliability). For example, power and

energy consumption as the *Internet of Things* (IoT) develops [8] is gaining more attention.

Hardware is the basis of all this domain and because of this, extensive work can be done directly on it to prevent or recover from faults directly on the hardware. This approach deletes most of the overhead of the software techniques, but it is much more restricted and complex than implementing and debugging a layer over a ”simple” hardware structure.

Moreover, this subject can be extended to *cybersecurity* in a way to harden the task (or at least the conception) of malicious programs [9]. In fact, an attacker could try to reverse engineer a program, he could also try to check the memory for data or the CPU usage and instructions or even using the power supply as an attack vector [10] to prevent a task from completing. For this purpose, several techniques have been developed to try and block either the static or dynamic analysis of a program [9]. Even measuring the redundancy of a program to be able to scale and compare two diversification techniques and choose one for example [11]. The research could even separate the low-level redundancy from a high-level, algorithmic, one.

Finally, for this thesis, the question is whether rewind on RISC-V processors for opaque enclaves in an unsecure world makes sense and can be implemented following the background work on SDRaD. It is interesting because the baseline answer is yes, but the cost it implies and the security it requires make the answer less obvious. This thesis studies rewind and discard, the hardware support around PMP and Keystone, the cycle overhead, and the cybersecurity properties that decide whether the mechanism is acceptable. It also looks at the conditions under which rewind remains useful and the conditions under which it becomes too expensive or too fragile. The reader should read this thesis to understand the practical limits of rewind, the assumptions it depends on, and the trade-off between recovery and overhead.

Furthermore, the codebase (implementation of the checkpoint and rewind enclave, host and tests) of this thesis can be found on the GitHub repository -. This is a fork of the main Keystone github Keystone repository [12] -. The files can also be found in the Appendix (8).

Chapter 2

State of the Art

For as long as computers (some could even say tools in general) have existed, so have defects, errors and bugs [2]. However, the need for a solid structure and concretes answers increased as Moore's law kept going on [13]. Luckily enough, this topic is drawing attention and a lot of ground work and precise basis have been established over the time.

2.1 Faults

Faults are the basis of the problem but the outcome of a fault, and the way it is handled, is equally important. To be clear and rigorous, we will use the Institute of Electrical and Electronics Engineers' (IEEE) [4, 14, 6] standards. In this goal, we can separate a fault and its consequences in 3 categories each one following from the previous one:

- **Fault**: A defect in a hardware device or component. It can be a *Single Event Upset* (SEU), otherwise seen as a bitflip or it can be other models such as *Multiple Event Upset* (MEU).
- **Error**: The difference between a computed, observed or measured value or condition and the true, specified, or theoretically correct value or condition. It is the effect of a fault on the system.
- **Failure**: The inability of a system or component to perform its required functions within specified performance requirements. In other words, the program malfunction due to an error.

Now, we need to categorize faults [4, 6]:

- **Transient**: Produces a temporary effect on the system in a very short period of time. It is often modeled using SEU. They are most likely cause by: -
  - alpha radiation (due to cosmic radiation or nuclear fission),
  - electromagnetic interference.
- **Intermittent**: Livespan similar to *Transient* but it occurs in *burst*. It is mainly caused by eternal conditions: temperature, voltage, hardware wear out and High Intensity Radiated-Field (HIRF).
- **Permanent**: Results from hardware wear out or manufacturing defects. The only fix for this type is to repair or change the hardware. Component wear out is well known in the current literature, the *Bathtub curve* [15] describes the three main phases of a component's life:

- The *infant* stage is when the hardware is fairly new (not in terms of design). Thus, it might be defective while still passing the basic tests required for using it. As this phase goes on, the average error rate decreases.
- The *main* stage is, for the main part, the longest one. The component is viable (since it has passed the previous phase) and not yet worn out. But still random faults occurs (due to environment, etc.). This phase is the best in term of fault rate (which is supposedly constant).
- The *wear out* stage is the first that comes to mind when speaking of permanent faults. A part of the component is defective and there is no direct fix except for replacing/repairing this part. In opposition to the first phase, the average error rate increases over time.

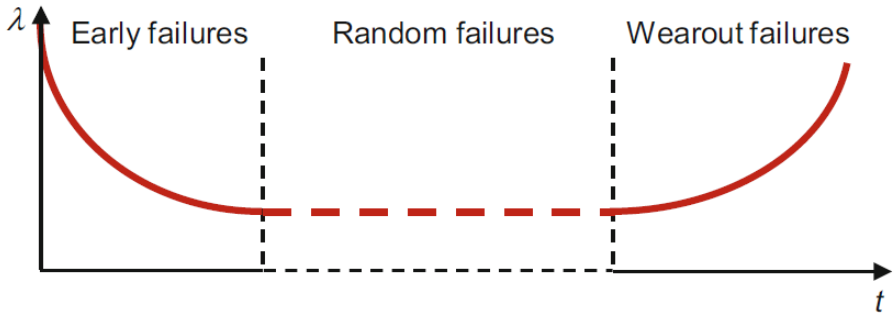

Figure 2.1: Bathtub curve. "Typical plot of the λ failure rate over time t (bathtub curve) with decreasing, constant, and increasing failure rate sections" - Figure 4.4 (p52) [16]

2.2 Fault Recovery Techniques

2.2.1 Space Redundancy

- **N-modular redundancy (NMR)**: this consists of N (2,3,4,...) instances of the job i running at the same time, when the job i finishes, a vote starts and the majority is returned as the result. NMR can detect by essence and correct the error. However, it is important to note that it can only detect an *error*, not a *fault* [4]. Also, each n instance of the job i should run on different cores to avoid SEU.

- **Standby redundancy**:

  - *Hot* standby is when the backup system runs in parallel with the main one. If something fails, the switch to the backup is almost instant since it is already active and synchronized.
  - *Cold* standby is when the backup system is off until the main one fails. When that happens, the backup starts up and takes over. This takes longer but saves resources while things are working.

2.2.2 Time Redundancy

- **Re-execution**: This technique runs the same task again to see if the error happens in the same way. If the second run works fine, the first one probably had a temporary problem. It

helps filter out transient faults. It has two main approaches:

  - *Backward error recovery*: If the state of the job or the environment needs to be restored after a fault, then the faulty job is restarted.
  - *Multiple recovery*: due to scheduling reasons, the currently preempted jobs are also re-executed even if not affected by the fault.

- **Checkpoint**: A checkpoint saves the state of the system at a certain time. If a failure happens later, the system can roll back to this saved state and continue from there without starting over from scratch. It is often used in High Performance Computing (HPC) for long-running tasks and in the industry in general as it is effective.
- **Recovery blocks**: This method runs a primary version of a task and checks the result. If the result fails, it runs another version (possibly degraded) to try again. This can repeat with more versions if needed until one gives a good output.
- **Forward error recovery**: Mostly used in networks, it executes a special routine that fixes the error without re-executing the original task. Simple raise/throw exception handling.

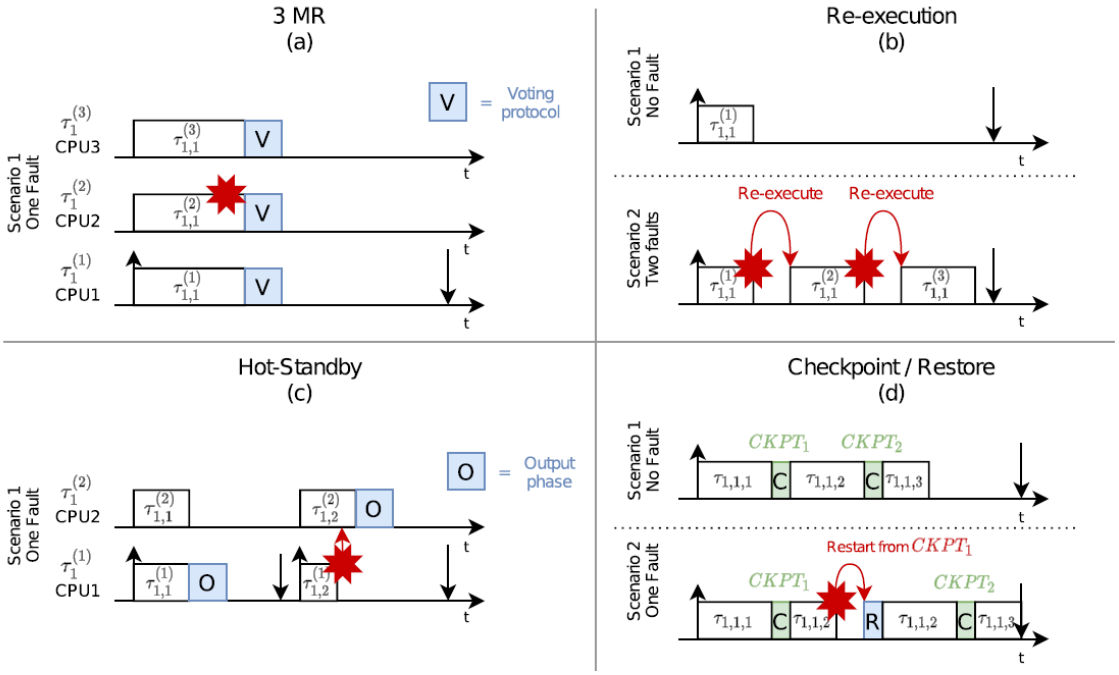

Figure 2.2: The four main fault recovery approaches.
"In (a), (b), and (c), the symbol $\mathcal{T}_{i,j}^{(k)}$ represents the j^{th} job of the i^{th} task, and k identifies the redundancy or re-execution job. Instead, in (d), $\mathcal{T}_{i,j,k}$ is the k^{th} part of the job $\mathcal{T}_{i,j}$. In the depicted example, the job $\mathcal{T}_{1,1}$ is composed of three parts" - Figure 1 of [4]

2.2.3 Fault Detection

Even though it is not the main topic, neglecting fault detection would be just as bad as neglecting any part of the domain. To highlight this, we can note that *NMR* does recover from an error occurring but it can *not* detect the underlying fault. Preventing the faults as soon as possible save

the whole recovery process and thus saving the additional computational power, possibly missed deadlines (in mixed-criticality systems for example) or even heat-induced wear out.

Hardware

- **Fail-signal processor** is used in multi-processor architectures. A processor complete the task and another one checks for abnormalities (time or result) on the first one.
- **Error detecting code** (EDC) are used to detect errors (for example checksum). It is used in the memory, the bus and on networks.
- **Error correcting code** (ECC) are used along EDC to try and correct small errors. They often need a matrix to be able to find the errors confidently.
- **Watchdogs** are used for the timing constraint. They ensure that is is respected.

Software

Hardware has the benefit of not affecting execution time because it works in true parallel. It can run checks or backups at the same time as the main task without slowing things down. For software, we have to consider the worst-case execution time of tasks along with the extra time used by the checks added to each job. Software can use multithreading to help, but the extra cost is still there.

2.2.4 Fault Injection

Designing and implementing a system without test cases is not imaginable. To this end, fault injection is being used. The idea is quite simple, put the system under various stress tests to check its tolerance to faults. This allows the use of a critical micro-processor with no more reluctance than necessary. These injections are made using four different techniques[9, 4].

- **Hardware**-based: Connect a device to mess with the system, for example with voltage spikes or imputing RF disturbances. A simple approach, yet effective.
- **Software**-based: Use a program to mess with the memory allocation of the system, for example allocating random or predefined segments. It is also fairly simple.
- **Simulation**-based: The hardware is *simulated* and faults are injected through registers using pseudo-random/predefined patterns. This gets more complicated.
- **Emulation**-based: The hardware is fully *emulated* by being written on a Field-Programmable Gate Array (FPGA). This allows host to inject faults in a transparent way (like a real fault would occur).

2.3 System's criticality

Critical systems are nearly everywhere today, from aviation [17] (DAL) to hospitals. A lot of the critical embedded systems are in the Internet of Things (IoT). To address such variety, we can use the Mixed-Citical models. These models consist of prioritizing the tasks with higher importance. For example, a task that needs to check if the distance with the car in front of you is large enough if more important than the one that checks the temperature. If the former fails, a crash might happen as if the latter fails, temperature might go up by 0.1°C. The formal way of doing this is to map

a number (the priority) to tasks, 0 being the most critical ones and as the number increases, the priority lowers.

The most common, to our knowledge, mixed-criticality model is the *Dual* [18]. It consists of two priorities:

- HI-critical
- LO-critical

As the names suggest, the HI-criticality stands for tasks that need to be finished in time (with a high priority) and LO-critical is used to describe tasks that can be dropped or can miss a deadline (low priority). The software failure model of a task is usually considered in two senses:

- the logical sense: the task produces an incorrect output,
- in the timing sense: the task does not produce a correct output by the end of its deadline.

However, all the industries (aviation, automobile, medicals, railway, spaces, industrials, etc.) have their own priority scale. All of them are, to some extent, a specification of a simple mixed-criticality system with task priorities. For example, aviation with *DO-178B* (or C) with the DAL priority that goes from A to E, A being the most critical and E being the least.

2.4 Cybersecurity

Cybersecurity is also a main point of attention in this field (as it is a field itself). This is even more valid with the growth of the IoT in general over this past decade and this growth increased with the new arrival of Artificial Intelligence (AI) [8]. This section will cover the basics of this project that we will use.

2.4.1 Threat Model

A threat model is a structured description of what must be protected, who may attack it, and which assumptions are accepted by the system. It helps separate the trusted part of the design from the untrusted environment and makes the security claims explicit. It is useful because the analysis depends on the untrusted parts, the trusted ones, the storage channel and the recovery path not having the same level of trust. It also makes clear which attacks are in scope, which assets must remain confidential or intact, and which limitations are acceptable for the chosen platform.

In this work, the threat model is especially important because the enclave must recover in an untrusted world while still preserving the checkpoint contents and the control flow of the recovery mechanism. This provides the basis for the later discussion of TEEs, enclaves, and the rewind protocol.

2.4.2 Trusted Execution Environments (TEE)

A Trusted Execution Environment (TEE) [19] is a secure part of the processor that runs code and handles data in isolation from the rest of the system. The goal is to protect sensitive operations from malware, faulty software, or even privileged system components like the OS or hypervisor. TEEs are often used for secure key storage, encryption, authentication, and digital rights management. It contains enclaves and is often run on *ARM TrustZone* for mobile devices [10].

2.4.3 Enclave

Enclaves provide isolated execution environments that protect sensitive code and data even if the Operating System (OS) is compromised [20]. They rely on hardware-level support (TEE) to keep the enclave memory hidden from other software, reducing the attack surface. Enclaves help contain faults, data and prevent them from spreading beyond their boundary, adding both security and resilience. It is used on mobile for example to provide a safe environment for banking, cryptographic or even biometric data storage. This is to avoid any third party software being able to access confidential data.

2.4.4 Code obfuscation & diversification

Code obfuscation and diversification make it harder for attackers to understand or predict the system's behavior. Obfuscation transforms the code into a version that works the same but is harder to analyze (static analysis). Diversification changes the memory layout, instruction set, or data locations across runs or devices. These techniques slow down attacks and can stop malware that depends on known patterns, making the system more fault-tolerant against targeted exploits (Zero-day vulnerability for example).

2.4.5 Rewinding & Domains

A current approach to address attacks is to first detect the attack and then roll-back the system to a previous (supposedly) safe state by replications or checkpointing. As much as this mechanism works to ensure reliability of the system, the *resilience* and *availability* when it is under an attack is not the focus. This last point enables an attacker to use the delay of roll-backing the system as an attack vector. To solve this, a software library has been created for 64-bits x86 processors: *Secure Domain Rewind and Discard (SDRaD)* [21]. The underlying logic consists of using *Domains*, compartmentalized and secured spaces and using a domain for each critical task. Thus, when an error occurs, the corrupted domain can be *discarded* (terminated) and the error can be handled. In this scenario, the target services are Commercials Off-The-Shelf (COTS) processors [21, 5, 4] (i.e. processors that don't need hardware specifications or specific changes in order to work) for web services with multiple clients. The plurality of the clients is where the danger lies as if one client is malicious, it can affect the others. That is where the SDRaD mechanism comes in place, each client or subroutines of the clients could be stored in different domains making a great isolation in-between clients. The program using the library can rewind the states of the domains discarded to avoid propagation and this makes the possible Denial of Service (DoS) attack outdated as the states are saved frequently and the whole system does not need to restart from scratch (using cache services for example).

2.4.6 Data & Control flow

Protecting data flow and control flow is key to stopping many types of (dynamic) attacks. Data flow checks make sure sensitive data does not leak or get overwritten. Control flow is the instructions order of execution for the CPU. Hardware or software guards can stop faults caused by memory corruption, buffer overflows, or instruction hijacking. Together, these methods help prevent faults from becoming full system breaches.

2.4.7 Obfuscation measurement

A derivation of this is the layout obfuscation, it consists only of destructuring the program. This is mainly done to prevent a human reading the code base and reverse engineering the whole system

(thus gaining informations and understanding of possible breaches). This is a one-way approach as when the structure is deleted (when classes are merged for example or comments and name deleted), it cannot be found afterwards. *Potency* and *Resiliency* are used to measure this [11], the former is to measure human readability and the latter is used to measure how well the system resists to automated decompilers/deobfuscators/deassemblers [9].

2.4.8 Limitations

One of the goals is to make the system more resilient to attacks and thus to rise the cost of developing attacks that could harm the system [9] (this is seen as control flow). Another one is to prevent someone with malicious intent to comprehend the system (static analysis) or even if they do, to prevent them understanding the flow of the program as it runs (dynamic analysis). However, as for every domain (in computer science), there are limitations. These inherent limitations are that no system is full proof.

2.4.9 Background work

The background work for this thesis is mainly built on Secure Domain Rewind and Discard. SDRaD introduces the idea of compartmentalized domains that can be rewound or discarded when an error occurs, so that faults do not necessarily force a full restart. That work is important here because it shows how rewind can be used not only as a recovery mechanism, but also as a way to contain faults and reduce the impact of denial-of-service-style failures in multi-client systems [21].

The second line of work is secure intermittent computing, where a system checkpoints its state, stores it outside the volatile execution context, and restores it after an interruption. The TrustZone-based work on Cortex-M shows that this can be done securely by protecting checkpoint confidentiality, integrity, authenticity, state continuity, and freshness inside a TEE, while keeping the recovery path small enough to remain practical. It also confirms that a checkpoint utility can be implemented with moderate runtime overhead on an embedded platform [10].

This thesis takes those ideas and places them in a Keystone setting. Instead of intermittent execution on Cortex-M, the focus here is on opaque enclaves on RISC-V, with checkpointing, sealing, and recovery handled by the enclave and the security monitor. The goal is therefore not to restate the prior work, but to adapt its security and recovery principles to a different architecture and to measure whether rewind remains practical under that model.

Chapter 3

Problem Statement and Objectives

Now that everything is, roughly speaking, well defined. Although the whole domain is very interesting and always needs specific studies on various topics, it needs to be narrowed down in order to suit a Master thesis.

First cut is the type of *redundancy*. For the purpose of the research, the tests will be based on the *timed* redundancy method that is *rewinding* against no redundancy. This allows to measure the viability and the relevance of the work done.

Second cut is the type of systems that needs to be modeled. Our focus will be *one level* criticality system, as they are great will reflect an enclave that has one single job. There could be extended work on several enclave levels that have a different saving rates. Making it mixed-criticalities enclaves a bigger system.

Third cut is the type of faults. As diving in any type of fault could be a thesis in itself, the faults will be reviewed only for what they are and not necessarily for their cause. Thus, *all three types* will be looked upon.

Next cut is the channel that will be used. In other terms, *hardware* and *software*. As they are both important and linked in various ways. The focus will not only, at least initially, be on one of them but rather on *both* using *emulation* and the implication that each has on the other. For example, Enclaves that need restricted access and create a waterfall of complexity in the design and implementation has an impact on both hardware and software. To this extent, we will base our work on *Keystone* that provides a platform to emulate a RISC-V processor with enclaves and cryptographic soundness using hardware (PMP) on a TEE[12, 22, 23]. This will be detailed in the corresponding Chapter 4.

Final cut is about the *cybersecurity* perspective. As the industry around this domain is mostly critical (aviation, automobile, space or even medical), there is a high risk with someone getting access to something they shouldn't have. Considering that, we must ensure that the faults occurring, possibly from a malicious source, do *not break the system nor give informations* about it. Not much from a static analysis (reverse engineer) perspective but rather from a *control flow* one. To make it more precise, on TEEs when a fault occurs, the system can try to changes of enclave as if it was a *Domain*. Then the system can be rewinded and set to a safe state while not being unavailable.

These specifications can be linked to try an extend the SDRaD [21] work in a TEE, making an equivalence between domains and enclaves. As the enclaves are subregions of a TEE, they are isolated and secure as the definition of a domain using Private Keys for Userspace (PKU) and

Memory Protection Keys (MPK) (with register mechanism and much more [21]).

Therefore, the research will focus on *Fault recovery by rewinding enclaves in a TEE on the RISC-V architecture using Keystone* and extending existing works. The main objective is to determine whether rewind is practical for opaque enclaves on Keystone when the world outside the enclave cannot be trusted. The first sub-objective is to measure the cost of checkpointing, loading, and ordinary execution so the break-even point can be identified. The second sub-objective is to verify that the recovery path still satisfies confidentiality, integrity and authenticity, atomicity, and unclonability. Together, these objectives answer whether rewind is useful in practice and what the implementation must assume to make it safe.

Chapter 4

Keystone Structure

”Keystone is an open framework for architecting trusted execution environments (TEEs) based on hardware enclaves. Keystone aims to provide an easy way of architecting and building a customized TEE for an arbitrary RISC-V platform with a simple workflow. This allows the TEE to be highly optimized for better performance, smaller trusted computing base (TCB), and programmability with a given workload and a threat model.” -2.2. Keystone Basics [23].

Its implementation can be found on the corresponding github repository: Keystone repository [12] .
The corresponding documentation can be found at: https://docs.keystone-enclave.org/ [22, 23].

4.1 RISC-V

RISC-V is a free and open-source (in contrast to x86 and ARM) Instruction Set Architecture (ISA). Due to its reduced instruction set, it has become a popular architecture for embedded systems and microcontrollers as it allows to reduce power consumption, code size, and memory use[24].

RISC-V has three privilege levels (2.1.2. RISC-V background [23, 25]):

- U-mode: User processes
- S-mode: Kernel
- M-mode: Bootloader-firmware

M-mode is the highest as it controls physical resources and interrupts. It is analogous to microcode in Complex Instruction Set Computer (CISC) ISAs (x86). However, unlike microcode, it is implemented in existing programming languages (C) and toolchains (gcc). This makes it much more accessible.

The running software can thus not be interfered or interrupted by lower modes. In Keystone, the M-mode is used for:

- the Security Monitor (SM)
- the Trusted Computing Base (TCB)

As all of these are software implementation. It is much easier to implement, debug and fix than it would be on hardware.

4.1.1 Physical Memory Protection (PMP)

"Physical Memory Protection (PMP) is a part of the RISC-V Privileged Architecture Specification which discribes the interface for a standard RISC-V memory protection unit."[26] It alows the M-Mode to control physical memory access. PMP is ruled by a set of Control Status Registers (CSR) to restrict physical memory access for U and S mode and is configured by the M-mode.

It also has a variable number of entries depending on the platform it is on. The one in our interest is QEMU, its virtual machine has 16 entries, each entry is defined by one or more CSRs. It works on physical addresses, the configuration of the subsequent virtual addresses can be let to the S-mode without compromising security. Even with different hardware implementations, the standard guarantees basic functionality.

More importantly, PMP is used by the SM to create memory isolation and can be dynamically configured by the M-mode (using CSRs).

4.1.2 Security Monitor (SM)

It is the root trusted software (and thus runs in M-mode). It is used to form the TCB and provides :

- Memory Isolation (PMP)
- Remote attestation
- Enclave thread management
- PMP synchronization
- Side-Channel Defense (FU540 module)

Being a software program, it must be verified first by hardware. This is done by the *Root of Trust* (RoT). At boot time, it first measures the SM, generates a keypair for remote attestation and then signs the public keep. After that, the boot is resumed.

The OS and the enclaves can call SM functions using the Supervisor Binary Interface (SBI). Each enclave uses a region (section) of memory, each of these regions are defined by PMP entries.

The SM uses by itself, two PMP entries (SM space, untrusted memory) each of the active enclaves consum one entry.

4.1.3 Interrupts and Exceptions

Any interruption or exception is received by the M-mode (by default). It thus has the full authority over the CPU scheduler. Some of its control can be delegated to S-mode. Using a lower level to catch exception such as page fault allows them to be handled quicker as the M-mode has a lot of workload. S-mode is great to handle page faults and system calls because they are frequent.

U and S modes access memory via virtual addresses while M-mode operates exclusively on physical addresses. In fact, the OS cannot tamper with the page table of the enclaves, this is because of PMP restrictions on the OS.

4.2 Basics

4.2.1 Overview

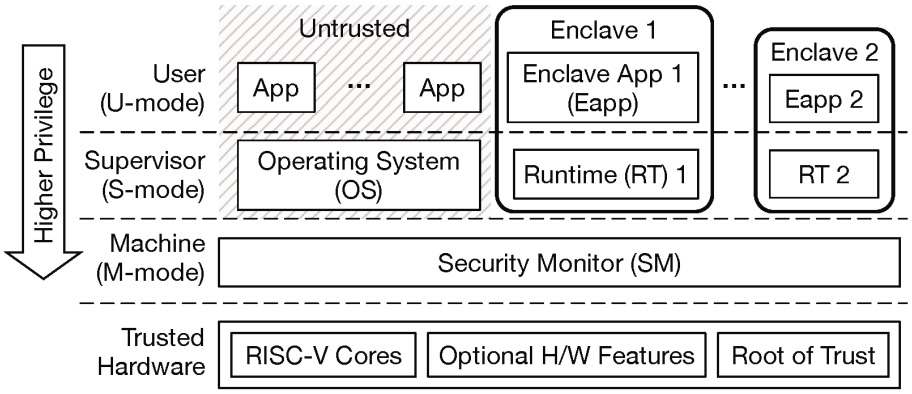

Figure 4.1: Keystone overview (2.2.1 [23])

The keystone environment is made of several, unequal and concurrent, components. The diagram shown in figure 4.1 highlights a privilege hierarchy, going from most to least privileged:

- Trusted Hardware: That is composed by *supposedly* trustworthy and contains RISC-V standards cores compatible with Keystone and a Root of Trust (for the SM machine). This part can also bring optional features working with memory (cache partitioning, encryption, cryptographic randomness, etc.). For these features to be usable, the SM requires platform specific plug-ins (FU540 for example).
- M-mode: The SM, with a small TCB is the communication between the OS and the Enclaves.
- S-mode: Composed mainly of the OS and then each RunTime (RT) part of enclaves is all separated to preserve isolation. The RT part of the enclaves manages all the system call, interruptions, memory managment and traps.
- U-mode: Composed by all the untrusted apps (supervised by the OS) of the computer and all the Eapp part of the enclaves. Note that the enclaves, even in this mode, are encapsulated.

4.2.2 Workflow

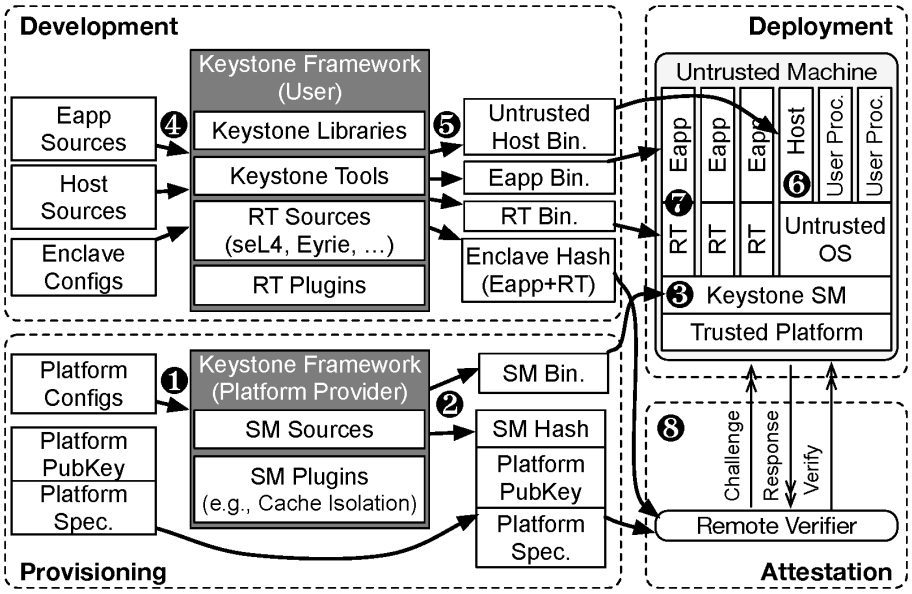

Figure 4.2: Keystone workflow (2.2.2 [22])

Keystone is a two-way framework. It is customizable by both the developers (of the enclaves) and by the platform producers. This means that building on it works in two different ways. The former is the one that is in our interest. Even more, the need to develop a host ”supervisor” of the enclave and interface to keep track in between runs will add a step.

The basic workflow is to build the eapps, the host and then the *possibly modified* runtime binaries. Then, they can (but don't have to), be packaged together into a file (4,5). The files are then deployed (6,7) into the remote system.

Keystone also allows the enclaves to do a remote attestation (2.2.2.3. [23]).

4.3 Enclaves

4.3.1 Enclave lifecycle

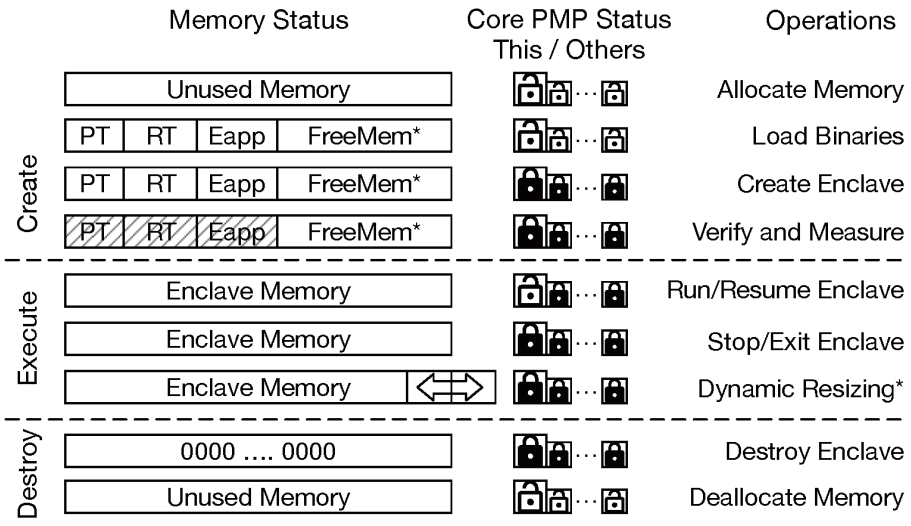

Figure 4.3: Enclave overall lifecycle (2.2.3 [23])

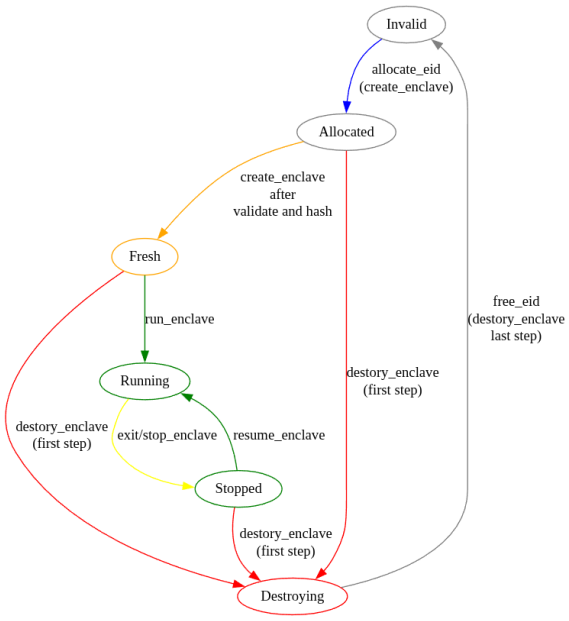

Figure 4.4: Current Enclave Lifecycle (1.9. Keystone Security Monitor[23])

There are three main states for the enclave to be in ((2.2.3) [23]):

- **Creation**: The (untrusted) host calls the SM with the Enclave Private Memory (EPM) allocated and the enclave's Page Table (PT). The SM then receives the host's call and encapsulate the EPM using a PMP entry. The entry is then shared among the cores so that the EPM is sealed from other cores. The SM then checks, right before execution, the state of the enclave (in case of an error).
- **Execution**: Still on the host side, it makes a call to the SM. This time to execute the enclave and pass it to one of the SM's core. The SM thus relieves a PMP entry and the execution starts. The enclave can be exited and resumed at any time in the execution. Each time the PMP permission is switched to maintain isolation.
- **Destruction**: As well as exiting and resuming the enclave, the host can destroy it. For that, the host calls the SM. The latter clears the EPM and then relieves the PMP entry. Last step is for the host to reclaim the newly available memory.

4.4 Quick EMUlator (QEMU)

QEMU is a free and open-source emulator and visualizer [27]. It has three different operating modes:

- The user emulation that runs a single program.
- The system emulation which emulates a (and possibly several) full system, including peripherals and different architectures like RISC-V. This is the mode used for Keystone.
- The hypervisor supports to act as a Virtual-Machine Manager (VMM) or a backend emulation-device for virtual machines.

4.4.1 Eyrie runtime

Eyrie is Keystone's reference enclave runtime, it runs inside the enclave in trusted mode, manages memory and basic system-calls. It sits between the enclave Eapp and the SM interface [28, 25]. The module is not a submodule of Keystone, this allows modularity and force manual updates [23].

4.4.2 FireSim

"FireSim is an FPGA-based cycle-accurate simulator for RISC-V processors. Using FireSim, you can test Keystone on open-source processors" (1.2.2 [23]). It is an alternative run (for Keystone's usage) to QEMU as it simulates the hardware [29].

4.5 Edge calls

An Edge call is a function that enters or exits the system. A visual representation is that it is at the *edge* of it, meaning that the method goes from inside to outside and the other way around.

In Keystone, it is used to communicate between the host (unsafe) and the enclave (safe). This is accomplished using shared memory between the two. More precisely using Outbound CALLs (`OCALL`). However, this is done by using simple offsets in memory region, neglecting virtual address pointers [23, 25].

4.5.1 OCALL lifecycle

To make things clear, we'll use an example of usage. If we want to print something from inside the enclave, we need to:

- First, decide, with the host (at implementation time), for the edge call corresponding to print: `OCALL_PRINT_VALUE`.
- Next, save the value input `val` ($enclave \rightarrow host$) we want to print.
- Optional, we decide of a return buffer value `ret` ($host \rightarrow enclave$).
- Now make the `OCALL`:

```
ocall(OCALL_PRINT_VALUE, &val, sizeof(val),
&ret, sizeof(ret))
```

Now the runtime part allocates an `edge_call_t` structure in shared memory. Then fills the call type and copies the value in the other part of the shared memory using an offset. The last part for the enclave now is for the runtime to exit the enclave without destroying it. This is done with an `SBI_CALL` expressing an ocall.

After that, the kernel resumes and checks for the enclave and sees a pending ocall. The execution is thus passed to the host. The host read the `edge_call_t` (destroying it at the same time) and notice that it is `OCALL_PRINT_VALUE`. The host then uses the same offset to read the buffered value and calls `_print_value` with the previously read value as an argument. After that, as for the enclave call, the `ret` buffer can be filled (if it was not 0 in the original ocall) with data in the shared memory. The `edge_call_t` status is set to `SUCCESS` and the edge call is done for the host.

An SBI_CALL is emitted and the runtime re-enters the enclave. The runtime checks for a returned value (if the return buffer was not 0 at the start). If so, it then uses the offset to put the data in ret and then resumes the wrapper code and passes any return value to the first ocall function.

Note that this can also be used to provide networking operations [25].

Since the implementation is asymmetric, the host will have to map the edge calls that it could receive to wrapper functions and be able to handle them while running the main loop. The enclave will send the OCALLs and wait for a possible response from the host.

Host-side

The edge call wrappers are working in parallel to the main loop. These wrappers are implemented as such:

```
// 1. Linking the edge calls to the functions
// this is done in configure_enclave()

// binds ocalls to the enclave
enclave.registerOcallDispatch(incoming_call_dispatch);
register_call(OCALL_PRINT_BUFFER,
            print_buffer_dispatch); // input
register_call(OCALL_SAVE_CHECKPOINT_BLOB,
            save_checkpoint_blob_dispatch); // input
register_call(OCALL_LOAD_CHECKPOINT_BLOB,
            load_checkpoint_blob_dispatch); // output

// 2. Basic edge call structure
void edge_call_wrapper(void *buffer)
{

    // 3. Checks the call sanity
    if (edge_call_args_ptr(edge_call, &arg_ptr, &arg_size) != 0
        || arg_size == 0)
    {
        edge_call->return_data.call_status = CALL_STATUS_ERROR;
        return;
    }

    /* 4. Wrapper logic */

    // 5. Setup return data
    ret_val = /* some logic */
    uintptr_t data_section = edge_call_data_ptr();
    memcpy((void*)data_section,
```

```
            &ret_val,
            sizeof(type_to_return));

    if (edge_call_setup_ret(edge_call,
                            (void*)data_section,
                            sizeof(type_to_return)))
    {
        call_status = CALL_STATUS_BAD_PTR;
    }

    // 6. Exiting
    edge_call->return_data.call_status = call_status;
    return;

}
```

Enclave-side

In contrast to the host, the edge calls in the enclave are sequential and in the main loop.

```
// The edge functions are :
// output
int save_checkpoint(uintptr_t stack_anchor, size_t anchor_len);
unsigned long ocall_print_buffer(char* data, size_t data_len);

// input
int load_checkpoint(struct rewind_checkpoint *checkpoint);


// 1. Basic edge call wrapper structure
auto edge_call_wrapper(int argc, char** argv)
{
    // 2. OUTPUT functions
    {
        /* output wrapper logic */
    }

    // 3. OCALL
    auto retval;
    ocall(OCALL_ENUM_NUMBER,
            data,
            data_len,
            &retval,
            sizeof(retval));
```

```
    // 4. INPUT functions
    {
        /* input wrapper logic */
    }

    //5. Exit
    return (output ? retvall: some_value);

}
```

Note that no standard library is used in the enclave as the Keystone tutorial (7.1.1 [22]) explicitly states that printf is not a secure I/O interface. Considering this, small functions to format numbers into strings (char*) have been implemented.

4.6 Data sealing

Keystone has a sealing protocol. Using this, an enclave can save data in an untrusted, non-volatile memory outside of its memory range. It works by enabling the SM to compute a `deterministic key` from the RoT for the enclave to use. The key works both for sealing and unsealing data. The deterministic operation is based on the device, a given enclave and a given SM running on a given processor will always yield the same key. Therefore, a combination including any differing component will give a different key [23, 25].

A simple example of usage is given in the github repository (Keystone repository [12]) at `examples/tests/data-sealing/data-sealing.c`:

```
char *key_identifier = "identifier";
struct sealing_key key_buffer;
int ret = 0;

/* Derive the sealing key */
ret = get_sealing_key(&key_buffer, sizeof(key_buffer),
                      (void *)key_identifier,
                      strlen(key_identifier));
```

This snippet is implemented in the enclave side. Note that there is an *identifier* , this allows the enclave to ask for several keys. Each of them is unique (and deterministic) for a given identifier.

Another subroutine for an enclave is to ask for an attestation using an SBI call `attest` that can be passed to the host using an edge call. The SM will generate and sign an attestation for the enclave. More informations about this can be found in the documentation at *5. Attestation* [23].

4.6.1 Key-Hierarchy

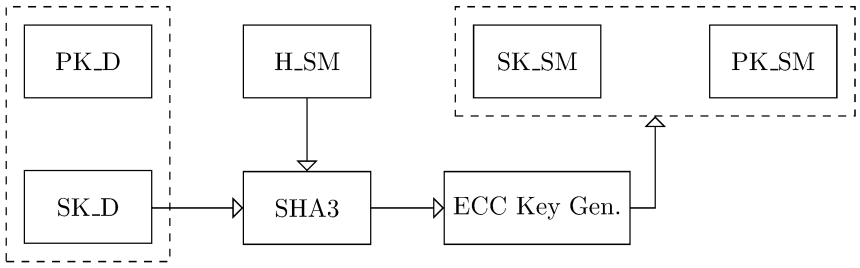

Figure 4.5: Key-Hierarchy (6.1. Keystone Key-Hierarchy [23])

The base of this hierarchy is at the left-most. This is the asymetric key pair (public and private) of the processor (`PK_D, SK_D`). Computing the private key with the SM hash (`H_SM`) which is derived from the RoT in SHA3. Thus, as long as the RoT is safe, so are the SM's measurrments. The result of the computation is used as a scalar for the Elliptic-Curve Cryptography (ECC) Key Generator. The final result is another key pair, the SM key pair (`PK_SM, SK_SM`).

4.6.2 Key Derivation

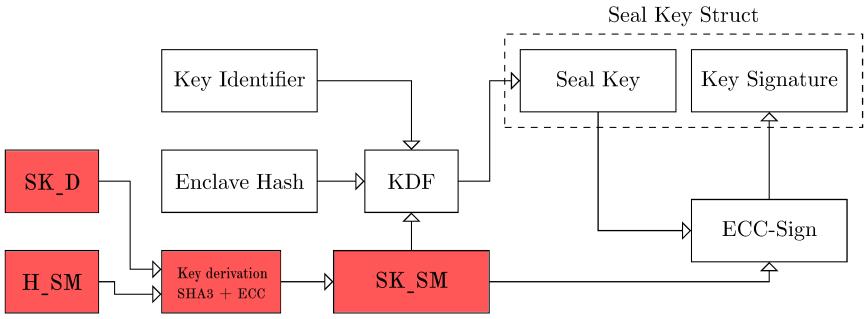

Figure 4.6: Key Derivation Diagram (6.2 Sealing-Key Derivation [23])

The red blocks are what we already computed in the, right above, Key Hierarchy 4.6.1 subsection. Now that the SM has a key pair, it can derive keys for the enclaves. This is done using three components:

- `SK_SM`: SM private key, it ensures that the key is bounder to the SM (and thus the RoT).
- The Enclave Hash, it ensures that the key is bounded to the enclave.
- A *key Identifier* (4.6), it is an additional input for the enclave to be able to derive multiple keys.

The three components are computed in a Key Derivation Function (KDF). The resulting sealing key is then signed using ECC and then placed in the Seal Key Struct (implemented as `sealing_key`):

```
struct sealing_key {
    uint8_t key[SEALING_KEY_LENGTH];
```

```
    uint8_t signature[SIGNATURE_SIZE];
};
```

4.7 Compilation

As shown in the documentation, compilation in Keystone is, at first sight, a simple task (1.2.1.3. Compile Sources [23]). However, all of its different, standalone, components (4.4.1, 4.4) make it very modular and thus more complex.

The basic command to compile the source files is:

```
make -j$(nproc)
```

This is divided into 3 different parts:

- `make`: Runs the `Makefile`.
- `-jN`: "Specifies the number of jobs (commands) to run simultaneously."-Manual page Ubuntu 20.04.
- `$(nproc)`: the `$()` is a shell substitution for the `N`, the output of the `nproc` command gives the number of available processing units and is placed in the `$()` substitution.

Altogether, the Makefile is executed using all the available processing units. This will build all the components and generate a bootable image, the output build in `build-$PLATFORM$BITS`. One can build it using the available options for it, these options are given below (presented with the default parameter) and can be given in the terminal using:

```
OPTION1=VALUE1 OPTION2=VALUE2 (...) make -j$(nproc)
```

4.7.1 Rebuilding Changed Components

We can include specific directories to be cleaned in order to avoid stale sources:

```
BUILDROOT_TARGET=<target>-dirclean
```

The new part is the targeted directory (`<target>`), this could be:

- `keystone-examples`
- `keystone-sm`
- `keystone-runtime`
- `all`
- etc.

When the compiler notices a stale, a warning message will be shown:

```
>>>  (WRN) Stale build directory detected for keystone-examples!
>>>  (WRN) You should build keystone-examples-dirclean to remove them.
>>>  (WRN) Not doing so may leave your build in an inconsistent state.
```

For this example, the following build command should be ran:

```
BUILDROOT_TARGET=keystone-examples-dirclean make -j$(nproc)
```

4.7.2 Platform

We can configure the platform to build. The default is `generic` and only supports qemu 4.4.

```
KEYSTONE_PLATFORM=generic
```

4.7.3 Bits

We can choose either RV32 or RV64.

```
KEYSTONE_BITS=64
```

4.7.4 Builroot Config file

Sets the configuration file for the buildroot to use.

```
BUILDROOT_CONFIGFILE=qemu_riscv$(KEYSTONE_BITS)_virt_defconfig
```

Chapter 5

System Model and Design

This chapter describes the concrete implementation, the choices and the assumptions that have been made.

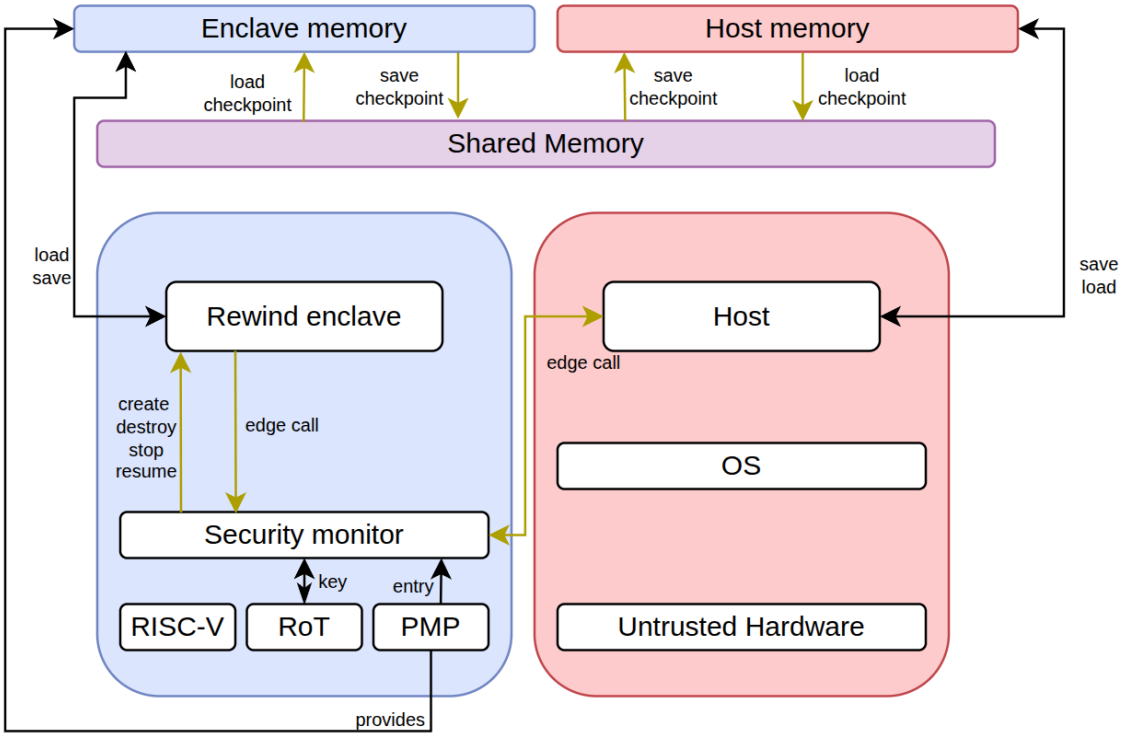

Figure 5.1: Rewind enclave architecture view

Our implementation is based on the Keystone ground work. This means that all of the enclave side (blue) is considered safe and all the outerworld (red) unsafe. The enclave never communicates with the host directly, the only shared space they have is the shared memory. This space is solely used to save and load the sealed checkpoint blob. The enclave then saves it, after the unsealing protocol, in its safe memory. The host supposedly does the same thing with its, unsafe, memory. The communication of the enclave is only based on the edge calls provided by Keystone. The host can also emit ones such as create, destroy, stop, resume. This indirect control of the host over the enclave is the reason that we cannot provide a roll-back and stall proof mechanism. More detailed explanations of each new part will be discussed in this section.

5.1 Preliminary setup and selection

The first step is to have a strong, yet modular, idea of the concept to be implemented. The goal is to obtain domains that can be checkpointed and the data that has been sealed needs to be accessible in another one. The domains discussed here are the ones from the "Rewind & Discard: Improving Software Resilience using Isolated Domains" [21]. We can make an easy parallel between enclaves and domains as they are executed in an isolated execution context with controlled memory (4.1.1) from the outer, untrusted, world. The SM (4.1.2) is an enclave factory and provides them with, as previously mentioned, controlled memory.

Furthermore, several enclaves of the same binary (compiled source file) can be initialized. This is important for the key generation of an enclave (4.6). In the case of a fault occurring in an enclave, the host can see the return code of the enclave and ask for the SM to create another instance of that same enclave (using PMP entries). The two instances will (as the key generation is deterministic [23, 25]) have the same resulting sealing key. This is true if we consider the identifier to be the same for both, as it is implemented in the binary of the enclave's eapp, the statement is supposedly satisfied.

For the checkpointing scheme, the enclave uses the edge calls (4.5) already implemented in keystone. The checkpointed data is sent and retrieved (to and from the host's memory space) using the enclave's key through two custom edge calls. To ensure that the data isn't tampered, there is a layer on top of Keystone's encryption scheme. The layer adds a tag and an Initialisation Vector (IV) for the enclave to check if the unsealed data corresponds to the sealed one. Only the last checkpoint is saved.

In this project, there is only one binary source (the compile code) and it doesn't implement any fault assessment, the enclave behaves as if the fault caused a fatal crash and stops immediately, returning an error. One possible fallback would be to use a degraded version of the enclave in order to prevent attacks using the computation power and overloading a server by making it fail a task indefinitely, for example. We address this in the future work section 7.2.4.

5.2 Threat Model

A threat model helps to clarify the hypothesis, the goals and the limitations compared to the real, unsafe, world. To this extent, we will ground the thesis over a specified one.

First, the trusted domain. It consists of the enclave code, its runtime, and its memory access under Keystone, RISC-V, and PMP isolation. It also includes enclave-derived sealing keys, obtained through `get_sealing_key`, because these keys are bound to the enclave measurement.

Second, the adversarial domain. This includes the **host**, the OS, and everything outside the enclave, with an exception for the *SM*, *RoT* and *PMP entries* that run the said enclave. It also includes the channel carrying the sealed blob, whether that channel is host storage, an OCALL buffer, or any other transport. Finally, it includes physical disruption such as power loss [21, 10], since an attacker may trigger crashes or power cuts.

Next, the attacker's capabilities. It can read, write, replay, or corrupt any sealed blob stored on the host. It can also trigger enclave stops, or even destruction, when incoming edge calls arrive, and it can crash power or execution at arbitrary points.

After that, the security objectives. They are confidentiality, integrity and authenticity, atomic-

ity, and unclonability. Confidentiality requires the checkpoint contents to remain unreadable to the host. Integrity and authenticity require that a tampered blob is detected and rejected without leaking information through response time analysis. Atomicity requires that no undefined state can arise from interrupted checkpoint writes. Unclonability requires that the device cannot be cloned from checkpoint data alone.

Finally, the accepted limitations. These are replaying and rollback are out of scope because the host is the only link to the outside world and cannot be trusted. Edge call manipulation, such as enclave stall or destruction, is also out of scope for the same reason.

5.2.1 Assumptions

The attacker may obtain arbitrary access to process (host) memory and may trigger run-time attacks that corrupt application state, but cannot modify code pages of the enclave under a W$\oplus$X policy [21, 30]. The attacker is also assumed to detect or exploit software-level vulnerabilities, while transient execution attacks, hardware-level fault injection, and other hardware or firmware-level attacks are out of scope and assumed to be covered by the Keystone security implementation. In the Keystone setting, we additionally assume the enclave isolation and the underlying secure world mechanisms protect enclave code and secrets against direct software manipulation, and that the host is not trusted with enclave state recovery details.

We further assume that the checkpointing and rewind mechanism operates securely, that is: a state external to the protected domain may be compromised, but the protected recovery boundary must remain trustworthy. For the intermittent-device context, the checkpointed state may be exposed to physical disruption such as power loss or tampering with external storage. The enclave only restores the checkpoint on restart and does not base its execution on the sealed checkpoint it sent unless there is a fault and thus a, possibly corrupted, rewind.

Enclave execution is assumed deterministic given identical entry state (no host-controlled inputs, per Attacker Capabilities). Under this assumption, IV/seq reuse across crash and restart is safe: identical plaintext re-encrypted under the same IV leaks nothing. This safety does not hold if the enclave itself introduces non-determinism (uninitialized memory, internal RNG/timer reads, etc.) before the point of divergence from a prior checkpoint_seq.

5.2.2 Requirements

The system must allow the application to continue operation after a detected fault or attack by restoring a trusted state and resuming execution from a safe point. The recovery mechanism must preserve memory integrity after rewind, and isolation must ensure that faults in one compartment do not compromise other protected compartments or the logic responsible for transitions and recovery.

An enclave state must: remain protected across enclave transitions, recovery metadata must be authenticated and, the host must not be able to forge or tamper with a checkpoint without detection.

5.3 Implementation

So far, this thesis has mainly covered concepts 2, ground work [10, 21, 12] and limitations. All of these are used to produce the implementation of the rewind enclave mechanism.

The files that are implemented and described can be found on the GitHub repository . Also

note that the repository setup is described at the end of the `README.md` (at the root of the repository itself). All of these files are also available in the Appendix 8.

Implementing the rewind example is done in two different places, the `eapp/` and `host` subfolders. The eapp is, as shown in the Keystone Basics 4.2, the trusted part and would be considered a *transient domain* [21]. A transient domain can exit (abnormally or not), it doesn't retain data upon exiting and is closed from the outer world (in both writing and reading). In comparison, the host would be considered a *persistent domain* as it keeps the enclave data (the sealed checkpoint), it cannot look into an enclave's memory region (except for the shared one given by the SM). However, the host is **not** trusted in itself.

To address this security issue, the checkpoint is sealed (using the sealing key provided by the SM) and uses a tag. This ensures that the host can neither read it or tamper with it. Upon unsealing a checkpoint, the enclave checks its tag so see if it fits the data. If not, the data is discarded and the enclave restarts the computation from the beginning.

The discard and rewind method is done by asking the SM to destroy the enclave, free the PMP entry and then allocate a new entry on the same binary to create the rewinded version of the enclave. This new enclave's instance will ask the host (using an edge call) for any saved, sealed, checkpoint. If the host has one it will copy the blob in the shared memory region and the enclave will consume it to try and restore the last checkpoint. If not, the enclave will start from scratch. Note that the first enclave also does this protocol (as the same code is being used).

Host-side

There is only one execution changing macro, other setups are the edge calls:

- `MAX_RUNS`: Decides how many tries the host should do before stopping and considering the job failed.
- `OCALL_PRINT_BUFFER = 1`: Links the OCALL 1 to the print (input) wrapper.
- `OCALL_LOAD_CHECKPOINT_BLOB = 8`: Links the OCALL 8 to the load checkpoint (output) wrapper.
- `OCALL_SAVE_CHECKPOINT_BLOB = 9`: Links the OCALL 9 to the save checkpoint (input) wrapper.

The rewinding protocol in the `host.cpp/.hpp` file 8 looks like:

```
// 1. setup:
//  - the macros in the CMakeList
//  - the edge calls number to be mapped

// 2. Setup the enclave parameters (memory-wise)
Params params;
uintptr_t ret = 1;
auto retry_counter = 0;
```

```
// 3. Beginning the execution loop
while (ret != 0  && retry_counter < MAX_RUNS)
{

    // 3a. Configure the enclave
    // enclave.init() && linking edge calls to
    configure_enclave(enclave, params, argv);

    //SM destroy the enclave upon completion
    enclave.run(&ret);
}

// 4. Exiting
return ret;
```

The basic edge call wrapper structure is explained in the edge call section of the keystone architecture 4.5.1. The actual implementation of each wrapper is detailed in 5.3.1.

Enclave-side (EAPP)

In contrasts to the host, there are two executions changing macro, other setups are the (same) edge calls. Note that the edge calls must be the same for the host and enclave in order to efficiently communicate.

- `EAPP_RUNS`: Decides how many iterations the enclave should do before stopping and considering the job done.
- `CHECKPOINT_INTERVAL`: Decides how many steps must be done in between two saves (minimal is 1).
- `OCALL_PRINT_BUFFER = 1` output
- `OCALL_SAVE_CHECKPOINT_BLOB = 9` output
- `OCALL_LOAD_CHECKPOINT_BLOB = 8` input

The rewinding protocol in the `rewind.c` file 8 looks like:

```
// 1. Setup:
//  - the macros in the CMakeList
//  - the edge calls number to be mapped

// 2. Load the structs
struct rewind_state state = {0, 1, 0}; // a, b, counter
struct rewind_checkpoint checkpoint;
struct fault_model f_m = get_default_model();

// 2a. check for an existing checkpoint save
```

```
if (load_checkpoint(&checkpoint) == 0)
{
    // 2b. try and restore the saved checkpoint if there is one
    restore_checkpoint(&state, &checkpoint)
}

// 3. Beginning the execution loop
for (; state.counter < EAPP_RUNS;)
{

    // Fault happens before the end of the computation
    // to mimic stall in the process.
    if (fault_should_trigger(&f_m))
    {
        // More explanations in Fault section
        EAPP_RETURN(16);
    }

    // Fibonacci
    auto next = state.a + state.b;
    state.a = state.b;
    state.b = next;
    state.counter++;

    if (state.counter % CHECKPOINT_INTERVAL == 0)
    {
        save_checkpoint((uintptr_t)&state, sizeof(state));
    }
}

// 4. Exiting
EAPP_RETURN(0);
```

5.3.1 Checkpoint

In order to rewind or discard an enclave, there must be a checkpointed state that is saved and loaded on demand (by the enclave, as defined by Keystone 4). As for the overall implementation, the two sides have their own wrapper logic as they have an asymmetric relationship. The host supervises the enclave and the latter is, once running, sealed from the former 8. However, as soon as the enclave submits an edge call, the host could (assuming malicious intentions) destroy the enclave or stall it indefinitely. This is bounded by Keystone structure and is, as noted in the threat model 5.2, not in the scope of this thesis.

One could implement a direct call from the enclave to the SM to external safe memory with, for example, table pages protected by outside PMP entry or another ”data enclave” that has a direct

communication channel with the ”runtime” one. The problem in our scenario is that the host is the only link to the outside world and is, by definition, also not to be trusted. This limitation being addressed, the saved checkpoint must be sealed and must be checked in order to not introduce unsafe data into the enclave's memory.

Moreover, below are two UML diagrams to visualize the lifecycle of the edge calls (OCALLs). An important point to notice is that the load lifecycle save lifecycle doesn't involve the enclave to read shared memory. The host cannot try to tamper it at this stage, but that is if we do not account the possible arbitrary `destroy()` call that the host could request to the SM. This would work as the SM sets the enclave's mode to `stop`, in this mode the enclave can be fully destroyed. This problem, however, is not in the scope of this thesis as it would need a total change in the Keystone design.

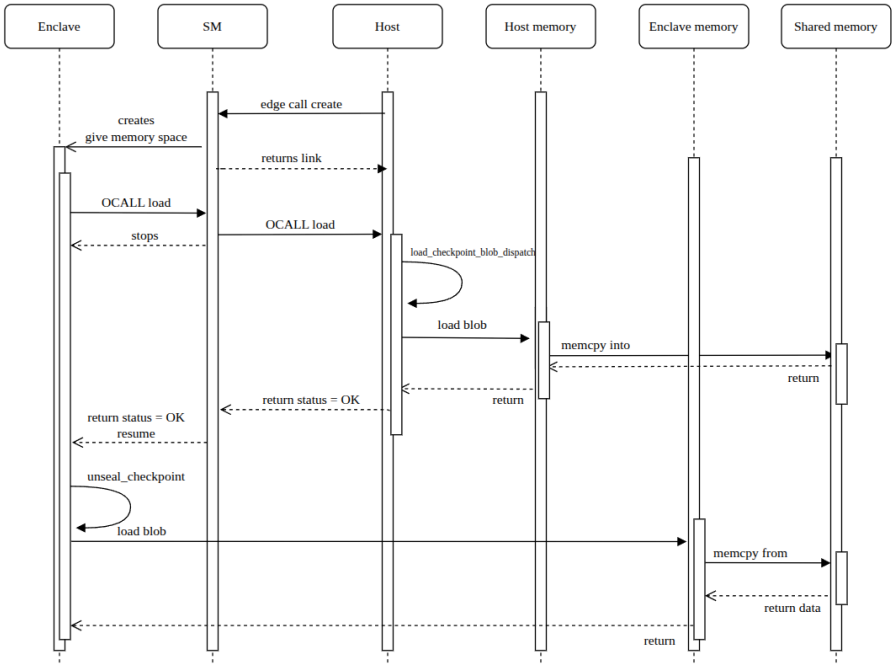

Figure 5.2: Overall view of an edge call life cycle to load the blob

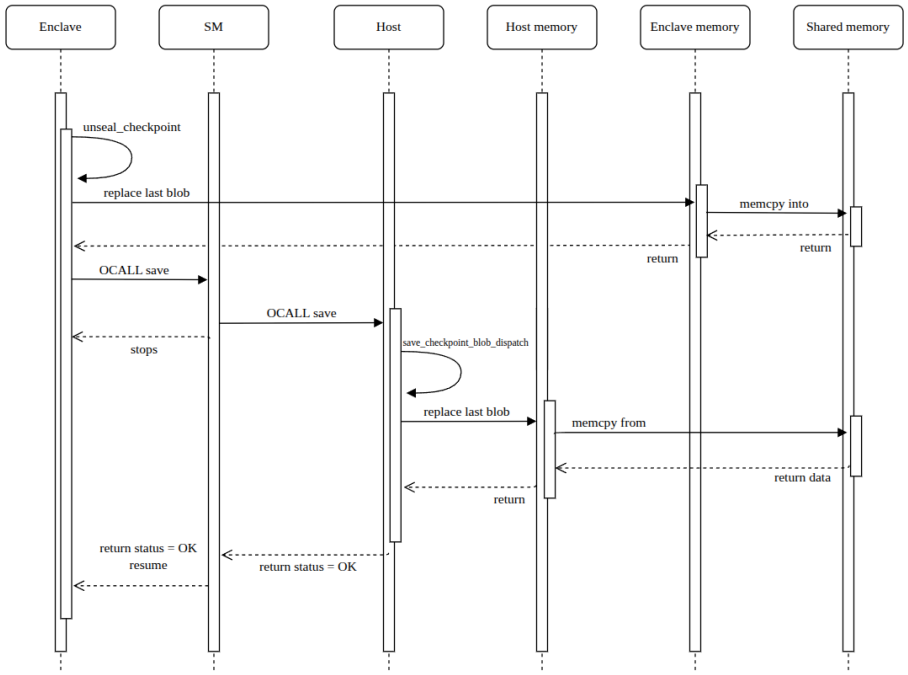

Figure 5.3: Overall view of an edge call life cycle to save the blob

Host-side

The print buffer will not be detailed as it is already implemented in the Keystone basic examples and is thus already explained in the documentation [22]. Furthermore, the host keeps the last saved checkpoint blob in the `inline vector<uint8_t> saved_blob` variable. This will be used for both loading and saving.

Notice that this variable is one of the possible attack vectors for a malicious program that would corrupt the host (in both reading and writing). The edge calls are, by definition and because they are linked to a specific function, another attack vector.

1. load wrapper

```
void load_checkpoint_blob_dispatch(void* buffer)
{
    // 1a. gather OCLL parameters
    struct edge_call* ecall = (struct edge_call*)buffer;

    // 1b. checks if there is a saved checkpoint
    if (saved_blob.empty())
    {
        ecall->return_data.call_status = CALL_STATUS_ERROR;
        return;
    }

    // 1c. looks into shared memory
```

```
    void* ret_buffer = (void*)edge_call_data_ptr();

    // 1d. save it
    memcpy(ret_buffer, saved_blob.data(), saved_blob.size());

    /* 1e. if TESTING enabled -> tamper */
    if (HOST_TESTING)
    {
        tamper_checkpoint_blob(saved_blob);
    }

    // 1f. return and exit with ok status
    edge_call_setup_ret(ecall, ret_buffer, saved_blob.size());
    ecall->return_data.call_status = CALL_STATUS_OK;
}
```

2. save wrapper

```
void save_checkpoint_blob_dispatch(void* buffer)
{
    // 2a. gather OCLL parameters
    struct edge_call* ecall = (struct edge_call*)buffer;
    uintptr_t arg_ptr;
    size_t arg_size;


    // 2b. check parameters sanity
    if (edge_call_args_ptr(ecall, &arg_ptr, &arg_size) != 0 ||
        arg_size == 0)
    {
        ecall->return_data.call_status = CALL_STATUS_ERROR;
        return;
    }

    // 2c. save (sealed) checkpoint
    saved_blob.resize(arg_size);
    memcpy(saved_blob.data(), (void*)arg_ptr, arg_size);

    // 2d. exit
    ecall->return_data.call_status = CALL_STATUS_OK;
}
```

Enclave-side

As for the host, the enclave must keep an anchor in order to be able to save/load a checkpoint. This is `extern struct rewind_state *state_anchor`.

1. load wrapper

```
int load_checkpoint()
{
    // 1a. sends an OCALL to the SM
    if (ocall(OCALL_LOAD_CHECKPOINT_BLOB,
            NULL,               // data to send
            0,                  // size of this NULL data
            &checkpoint_blob,   // data to retrieve
            sizeof(checkpoint_blob)) != 0)
    {
        eapp_print("No saved checkpoint");
        return -1;
    }

    // 1b. try to unseal the retrieved data
    if (open_checkpoint_blob(&checkpoint_storage,
                        &checkpoint_blob) != 0)
    {
        return -1;
    }

    // 1c. increment sequence and exit
    /*
    the sequence is incremented to mimic what would happened
    if the enclave was at this iteration and just saved
    (the increment is done after the save).
    */
    checkpoint_storage.checkpoint_seq++;
    return 0;
}

// Note: The enclave must then use
// restore_checkpoint()
// to restore the decrypted checkpoint into the state_anchor
```

2. save wrapper

```
int save_checkpoint()
{
```

```
// 2a. setup parameters
uintptr_t stack_sp;
uintptr_t stack_anchor;
uintptr_t snapshot_sp;
uintptr_t snapshot_end;
size_t snapshot_len;
struct checkpoint to_save;  // temporary built checkpoint

// 2b. load and check the parameters
if (state_anchor == NULL)
{
    eapp_print("Invalid checkpoint anchor");
    return -1;
}

memset(&to_save, 0, sizeof(to_save));
stack_sp = read_stack_pointer();
stack_anchor = (uintptr_t)state_anchor;
snapshot_end = stack_anchor + sizeof(*state_anchor);

snapshot_sp = snapshot_end > STACK_SNAPSHOT_SIZE
    ? snapshot_end - STACK_SNAPSHOT_SIZE
    : stack_sp;

// keep the snapshot aligned to the current stack
if (snapshot_sp < stack_sp) {snapshot_sp = stack_sp;}

if (snapshot_end < snapshot_sp)
{
    eapp_print("Invalid snapshot anchor");
    return -1;
}

snapshot_len = snapshot_end - snapshot_sp;

if (snapshot_len > STACK_SNAPSHOT_SIZE)
{
    eapp_print("Stack snapshot too large");
    return -1;
}

// 2c. save the current data into the tmp variable
to_save.checkpoint_seq = checkpoint_storage.checkpoint_seq;
```

```
    memset(to_save.stack_data, 0, sizeof(to_save.stack_data));
    memcpy(to_save.stack_data+(STACK_SNAPSHOT_SIZE-snapshot_len),
            (void *)snapshot_sp,
            snapshot_len);

    memset(&checkpoint_blob, 0, sizeof(checkpoint_blob));


    // 2d. seal blob
    if (seal_checkpoint_blob(&checkpoint_blob, &to_save) != 0)
    {
        return -1;
    }

    // 2e. output OCALL
    if (ocall(OCALL_SAVE_CHECKPOINT_BLOB,
            &checkpoint_blob,   // data to save
            sizeof(checkpoint_blob),
            NULL, 0) != 0)  // no data to wait from the host
    {
        eapp_print("failed to save checkpoint");
        return -1;
    }

    // 2f. increment checkpoint_seq and exit
    checkpoint_storage.checkpoint_seq++;
    return 0;
}
```

5.3.2 Data encryption

To preserve the security objectives, the enclave implements a cryptographic protocol Appendix 8.

The authentification and encryption keys are both derived using the function `derive_checkpoint_material()` usging Keystone `get_sealing_key()` 4.6. This function is cached after the first usage as the keys will never be changed afterwards. The workflow can be described as such:

1. Save

a) `derive_checkpoint_material()` asks the SM sealing API for key material bound to scheme label (`identifiers[]`).

b) The returned material is two independent AES-256 keys:
- `enc_key` for confidentiality (CTR stream encryption)
- `auth_key` for integrity/authenticity (CBC-MAC)

c) `encrypt_stack(payload)` AES-CTR-encrypts the plaintext payload and the iv.

d) `compute_tag()` MACs the ciphertext checkpoint payload.

e) The payload and tag are copied into the host-facing blob.

```
// 1a-b. Derive checkpoint key material
if (derive_checkpoint_material() != 0) { return -1; }

// 1c. encrypt_stack(payload):
// AES-CTR-encrypt payload, build iv from checkpoint_seq
uint8_t iv[AES_BLOCK_SIZE] = {0};
memcpy(iv,
    &checkpoint->checkpoint_seq,
    sizeof(checkpoint->checkpoint_seq));
memcpy(iv + sizeof(checkpoint->checkpoint_seq),
    &checkpoint->checkpoint_seq,
    sizeof(checkpoint->checkpoint_seq));

uint8_t mac_input[AES_BLOCK_SIZE +
                  sizeof(struct checkpoint)];
size_t mac_len = AES_BLOCK_SIZE + payload_len;
memcpy(mac_input, iv, AES_BLOCK_SIZE);
memcpy(mac_input + AES_BLOCK_SIZE, payload,
    payload_len);


aes_key_setup(enc_key, enc_schedule, AES_KEY_BITS);
aes_encrypt_ctr(mac_input, mac_len,
            mac_input,
            enc_schedule,
            AES_KEY_BITS,
            iv);

// 1d. compute_tag(): AES-256-CBC-MAC over the ciphertext and iv
uint8_t zero_iv[AES_BLOCK_SIZE] = {0};
aes_key_setup(auth_key, auth_schedule, AES_KEY_BITS);
aes_encrypt_cbc_mac(payload, payload_len,
                    tag,
                    auth_schedule, AES_KEY_BITS,
                    zero_iv);

// 1e. Copy ciphertext payload, iv and tag into the blob
memcpy(blob->iv, iv, sizeof(iv));
memcpy(blob->sealed, payload, sizeof(payload));
```

```
memcpy(blob->sealed + sizeof(payload), tag, sizeof(tag));

// the blob is then sent to the host using the
// previously described OCALL_SAVE_CHECKPOINT_BLOB edge call
```

2. Load

a) Derive the same `enc_key`/`auth_key` again from the same context.

b) Split the opaque blob into ciphertext, iv, and tag.

c) Recompute the expected tag over the ciphertext and given iv.

d) Compare expected tag with the stored tag, when mismatch, reject.

e) Decrypt the ciphertext.

f) Rebuild the plain checkpoint.

```
// 2a. Derive the same enc_key/auth_key
// again from the same context
if (derive_checkpoint_material() != 0) { return -1; }

// 2b. Split the opaque blob into ciphertext, iv, and tag
memcpy(iv, blob->iv, sizeof(iv));
memcpy(ciphertext, blob->sealed, sizeof(ciphertext));
memcpy(tag, blob->sealed + sizeof(ciphertext), sizeof(tag));

// 2c. Recompute the expected tag over the ciphertext
compute_tag(ciphertext, sizeof(ciphertext), expected_tag);

// 2d. Compare expected tag with the stored tag;
// reject on mismatch
if (memcmp(expected_tag, tag,
           sizeof(expected_tag)) != 0) {
    eapp_print("checkpoint authentication failed");
    return -1;
}

// 2e. Decrypt the ciphertext,
// same iv derived from checkpoint_seq
aes_key_setup(enc_key, enc_schedule, AES_KEY_BITS);
aes_decrypt_ctr(ciphertext, sizeof(ciphertext),
                payload,
                enc_schedule, AES_KEY_BITS,
                iv);
```

```
// 2f. Copy the plain decrypted checkpoint
memcpy(checkpoint, payload, sizeof(*checkpoint));
```

Data structures

Three data structures separate three distinct roles in the checkpoint/rewind pipeline:

```
struct checkpoint checkpoint_storage
struct sealed_checkpoint checkpoint_blob
struct rewind_state *state_anchor
```

The implementation size of the resulting blob can be seen by setting the macro (and the overall eapp tests) `EAPP_BLOB_SIZE_TESTING` to 1.

Component	bytes
IV	16
Plain text	8208
Tag	16
Total	8240

Table 5.1: Sealed checkpoint size (bytes)

- `struct checkpoint checkpoint_storage`: the *plaintext* in-enclave representation of the saved state (e.g. register file, stack snapshot, sequence number). This is the object produced right before sealing and the object reconstructed right after unsealing. It never leaves the enclave in this form.
- `struct sealed_checkpoint checkpoint_blob`: the *host-oriented, opaque* container holding the encrypted payload, the IV, and the authentication tag. This is the only representation of the checkpoint that is allowed to cross the enclave boundary and be stored/-transmitted by the untrusted host.
- `struct rewind_state *state_anchor`: the *runtime pointer/metadata* used by the rewind mechanism to locate and validate which checkpoint(s) are eligible for restoration (e.g. current sequence number, pointer to active checkpoint, rewind policy state). It is not itself sealed, it governs *which* `checkpoint_blob` may be opened and *when*.

Cryptographic primitives and concepts.

- **CBC-MAC**: Message Authentication Code built by running block-cipher CBC encryption over the message and keeping only the final block as the tag. Provides integrity/authenticity, *not* confidentiality. Secure only for fixed-length messages unless additional length-binding is used.
- **CTR (Counter mode)**: block-cipher mode that turns AES into a stream cipher by encrypting successive counter values and XOR-ing the result with plaintext/ciphertext. Provides confidentiality only; requires a unique `(IV, key)` pair per encryption to remain secure.

- **ENC / AUTH**: shorthand for the two independent purpose-bound keys derived from the sealing key material: `enc_key` (used only for CTR confidentiality) and `auth_key` (used only for CBC-MAC integrity). Key separation prevents cross-purpose attacks between encryption and authentication.
- **IV / Nonce / Seq**: Initialization Vector (or *nonce*, "number used once") is the per-encryption input that randomizes/varies the keystream so identical plaintexts don't produce identical ciphertexts. Here it is derived deterministically from `checkpoint_seq`, a monotonically-assigned sequence number identifying each checkpoint instance. Its uniqueness across all encryptions under the same key is the core security requirement of CTR mode.
- **TAG**: the output of the CBC-MAC over the ciphertext. i.e. a fixed-size value that the verifier recomputes and compares against the stored value to detect any tampering with the sealed payload.

5.3.3 Fault Modeling

Advanced fault detection and correction is not in the scope of this thesis. Thus, a simple fault model is implemented: Appendix 8. It is based on the SplitMix [31, 32] algorithm. It is a small, object-oriented, non-cryptographic, Pseudo-Random Number Generator (PRNG) and is based on Linear Congruential Generators (LCG) followed by bit-mixing (XOR-shifts) operations. It produces good enough results while being lightweight.

We maintain a 64-bit (SplitMix64) `fault_model` structure with three fields:

- **seed**: initial value for the generator
- **step**: current iteration count
- **period**: inverse of fault probability
- **count**: number of faults that have occurred

The seed and the period can be chosen by modifying the corresponding macros in the `CMakeLists.txt` 8. Furthermore, the seed can be either random (based on the runtime cycle counter) or fixed.

- `SEED`: The seed to be picked, useful for testing.
- `FAULT_RANDOMIZE_SEED`: If set to true (by default) then the seed is pseudo-randomly picked (using the cycle counter), otherwise it is set to `SEED`.
- `PERIOD`: The number of calls to the function, on average, before it should return true.

The normal usage is:

```
// 1. setup the macros in the CMakeList

// 2. initialize the fault_model
struct fault_model f_m = get_default_model();

// 3. at each step, trigger a possible fault
```

```
// chances are 1/PERIOD
if (fault_should_trigger(&f_m))
{
    // 4. Exit then enclave with abnormal code
    // Keystone::Error::EnclaveInterrupted = 16, the enum
    EAPP_RETURN(16);
}
```

Note that we treat a fault as an error. That is because the goal is to checkpoint and not to catch (or to directly correct) SEUs, bursts or even permanently damaged (worn out) hardware. Malicious data tampering in the enclave is not discussed as Keystone supposedly guarantees data isolation using PMP 4.1.1, the SM 4.1.2 (and thus a RoT). Moreover, other possible fault modeling approaches are discussed in 7.2.3.

5.3.4 Scripts

Building and running scripts are great to make the development quicker and less prone to error. There are three scripts at the root of the project that can be used in order to compile, to run and to do both sequentialy:

- `make-examples.sh`: Serves as the compilation (of the examples 4.7).
- `run-rewind.exp`: The most complex one as qemu 4.4 is an emulation, inputs and waiting times (interactions) are expected from the user. Thus, a `.exp` (expect) script is perfectly suited.
- `make-run.sh`: A simple concatenation of the other two.

The **logs** produced can be found in the `build-$PLATFORM$BITS/build.log` file.

Chapter 6

Experimental Setup and Evaluation

This chapter measures the cost of rewind and the security effect of the recovery path. It measures checkpoint save cost, checkpoint load cost, ordinary compute cost, and checkpoint size. It also checks whether the implementation still satisfies confidentiality, integrity and authenticity, atomicity, and unclonability. These measurements answer the question of practicality because they show when checkpointing is cheaper than restarting from scratch. They also address the goals and objectives because they test whether the host can stay untrusted while the enclave still recovers safely.

6.1 Macros

The tests are being conducted using different macros that are all defined in the `CMakeList.txt` following this structure:

```
## Definition, the "some_value" can be left to ""
set(EXAMPLE_MACRO "some_value" CACHE \
     STRING "explanation of the macro")

## Link for the eapp to access it
if(NOT "${EXAMPLE_MACRO}" STREQUAL "")
  target_compile_definitions(${eapp_bin} \
    PRIVATE MACRO_TO_BIND=${EXAMPLE_MACRO})
endif()

## Link for the host to access it
if(NOT "${EXAMPLE_MACRO}" STREQUAL "")
  target_compile_definitions(${host_bin} \
    PRIVATE MACRO_TO_BIND=${EXAMPLE_MACRO})
endif()
```

In case of them not being defined or if the field is left as `""` in the `CMakeList.txt`, a fail safe is there in each relevant header file using this structure:

```
#ifndef EXAMPLE_MACRO
#define EXAMPLE_MACRO some_value
#endif
```

There are various ones, **three main categories** emerge:

- **Logging**, to enable/disable output from host/enclave.
- **Execution**, to control the workflow (the number of computation iterations and tries).
- **Testing**, to enable/disable tests from the host/enclave.

6.1.1 Logging

These macros are not to be confused with testing macros as they are independent from them. One could run the tests of the side with the logging of the enclave or all the tests with no logging (which wouldn't be very interesting).

- `HOST_LOGGING`: enable host printing
- `EAPP_LOGGING`: enable eapp printing
- `EAPP_BREAK_EVEN_CSV_OUTPUT`: output a csv format in order to produce graphs (6.5)

6.1.2 Execution

- `EAPP_RUNS`: sets the number of iterations the enclave does
- `HOST_MAX_RUNS`: sets the maximum number of try before stopping
- `FAULT_PERIOD`: sets the inverse of the chances to trigger a fault per call
- `CHECKPOINT_INTERVAL`: sets the period the checkpointing, i.e. the number of steps to do in between saves

6.1.3 Testing

Because the two parts are opaque to each other, tests must be implemented on both sides. These macros enable (or disable) the tests in a branching way.

Host-side

- `HOST_TESTING`: launches the host in testing mode, reduces outputs and run `ANALYSIS_RUNS` iterations to compute averages
- `ANALYSIS_RUNS`: sets the number of runs for the analysis (host side)
- `TEST_TAMPER_MODE`: set the mode of tampering test to do (none/bitflip/truncate/last-byte)

Enclave-side

- `EAPP_TESTING`: launches the execution in testing mode, starts the test sequence
- `FAULT_RANDOMIZE_SEED`: enable/disable random seed for the fault model
- `FAULT_SEED`: sets the seed to start from for the fault model (if `FAULT_RANDOMIZE_SEED`=0)

- `EAPP_AVG_FAULT_TESTING`: enables the fault model average period testing
- `EAPP_BLOB_SIZE_TESTING`: enables the blob size benchmark
- `EAPP_ROUND_TRIP_TESTING`: enables the round trip test
- `EAPP_CYCLE_BREAKDOWN_TESTING`: enables the cycle cost breakdown benchmark
- `EAPP_BREAK_EVEN_TESTING`: enables the break-even threshold benchmark

Manipulating the macros can get confusing as they are independent. One could launch a full set of host testing with `HOST_TESTING`=1 and `ANALYSIS_RUNS`=100 (to analyze the average runtimes, for example) while enabling the `EAPP_TESTING` making the enclave only executing tests (and even more, returning failed tests that the host will try to accomplish). Or trying tests like `TEST_TAMPER_MODE`=bitflip and not setting the `HOST_TESTING` to 1. In order to better understand what is going on, it is recommended to enable both logging macros and disable the csv one.

6.2 Test definitions

This section defines the test metrics that we are looking for to evaluate the checkpoint and rewind mechanism. There are two parts: the *analysis*, where we quantify execution costs and overhead, and the *cybersecurity*, where we validate the threat model assumptions and security objectives from 5.2.

The metric that is in our main interest for the *analysis* is the **cycle cost** taken for each of the main operation of the enclave. These main operations are: *sealing* the checkpoint, *unsealing* it and the *computation* made by the enclave at each step. The goal is to find when does the use of rewinding exceed the costs of using it. We also will compare the data cost per byte, for these operations to see what would it cost to be using other sizes for different payloads.

The second metric used to compute this given cycle cost and the corresponding threshold to make the rewinding viable is the **Number of errors**. We will fix this number for a given run and check what is the gain of the rewind over starting from scratch.

After that come the, previously stated, main operations. A naive enclave would only do the *computation* at each step but a Rewind is more complex and costs more to operate. We thus need to fix a size for each of them to compare the costs and when they can balance out each other.

Finally, we will look into the optimal saving interval to be used considering the other parameters.

Now, for the *cybersecurity*, we will assess that the security objects are, along with the requirements, respected and fulfilled.

6.3 Results

This section interprets the measurements collected from the tests defined above. It derives the checkpointing cost model, compares the model with the runtime benchmarks. It also checks the security assumptions against the threat model and the stated objectives.

6.4 Static Analysis

The enclave is made to run in discrete time, for our calculations, we will use:

- N the number of runs it is doing before the job is considered done
- K the number of occurring faults during the enclave runs
- $errors$ a set of size K of the indexes of the occurring faults
- C_{event} the total number of **cycles** used by the *event* (save, load, compute, etc.)

Having these variables states we can compute the theoretical cost of checkpointing compared to restartarting the enclave from scratch. A small nuance is that the C_{save} and C_{load} do not include the corresponding edge calls because of their asymmetry.

$$C_{checkpoint} = N \cdot (C_{save} + C_{compute}) + K \cdot C_{load} \tag{6.1}$$

$$C_{no\_checkpoint} = (C_{compute}) \cdot (N + \sum_{i=0}^{K}(errors[i]))$$

Note that the sum of the error indexes ($\sum_{i=0}^{K}(errors[i])$) as an expected value of $K \cdot \lfloor \frac{N}{2} \rfloor$.
This is because the probability of a fault occurring at index i over $[0, N]$ (X_i, which is equal to $errors[i]$) is $\frac{1}{N}$ as X_i is iid. Thus:

$$\mathbb{E}[X] = \frac{N}{2} \tag{6.2}$$

The average (expected value) can be derived using basic probability:

$$\begin{aligned}
\mathbb{E}\left[\sum_{i=0}^{K} \text{errors}[i]\right] &= \mathbb{E}\left[\sum_{i=0}^{K} X_i\right] \\
&= \sum_{i=0}^{K} \mathbb{E}[X_i] \\
&= \sum_{i=0}^{K} \mathbb{E}[X] \quad \text{(since } X_i \text{ are i.i.d.)} \\
&= K \cdot \mathbb{E}[X] \\
&= K \cdot \frac{N}{2}
\end{aligned}$$

One can test the validity of this statement in the implementation by running the average fault testing and the break-even threshold.
Using that result, $C_{no\_checkpoint}$ can be written as:

$$C_{no\_checkpoint} = C_{compute} \cdot \left(N + \frac{K \cdot N}{2}\right) \tag{6.3}$$

6.4.1 Computation threshold

Having the equations 6.1 and 6.3, we can compute the threshold. This threshold is obtained when the ratio shown below is **greater or equal** to **one**.

However, the equations below are taking the *expected* values (average), a single iteration of an enclave of this will **not** follow this exact rule. This is because of the randomness, a simple (yet more than rare) example would be that an iteration with 10 faults have all of them at the first step. The number of runs for $C_{no\_checkpoint}$ is then $N + K$ which is not only very much below the average but also equal to $C\_checkpoint$. The latter being strictly more expensive in cycles will perform more poorly. The expected behavior for a given K is discussed in Section 6.1.

$$ratio = \frac{C_{no\_checkpoint}}{C_{checkpoint}} \tag{6.4}$$

$$ratio = \frac{C_{compute} \cdot \left(N + \frac{K \cdot N}{2}\right)}{N \cdot (C_{save} + C_{compute}) + K \cdot C_{load}} \tag{6.5}$$

$$\frac{C_{compute} \cdot \left(N + \frac{K \cdot N}{2}\right)}{N \cdot (C_{save} + C_{compute}) + K \cdot C_{load}} \geq 1$$

$$C_{compute} \cdot \left(N + \frac{K \cdot N}{2}\right) \geq N \cdot C_{save} + N \cdot C_{compute} + K \cdot C_{load}$$

$$C_{compute} \cdot \left(N + \frac{K \cdot N}{2} - N\right) \geq N \cdot C_{save} + K \cdot C_{load}$$

$$C_{compute} \cdot \frac{K \cdot N}{2} \geq N \cdot C_{save} + K \cdot C_{load}$$

$$C_{compute} \geq \frac{N \cdot C_{save} + K \cdot C_{load}}{\frac{K \cdot N}{2}}$$

The final equation is thus:

$$C_{compute} \geq \frac{2\,C_{save}}{K} + \frac{2\,C_{load}}{N} \tag{6.6}$$

This confirms that what is seen in the graphs of Section 6.5:

- For $k = 0$, the model can only approach, from below, the threshold ratio.
- As the C_{save} and C_{load} are rising, the $C_{compute}$ must also go up, this translates to a plateau after the first threshold overpass.
- The number of fault K, that is supposedly much smaller than the number of runs N, which is great ($\geq$ 10.000) as per the goal of long-running tasks, makes the C_{load} weight little compared to C_{save}

Given the actual costs of the save and load cycles as shown in 6.5.2. We can isolate the computation

needed in order to reach the threshold value.
Assuming that we use:

- $N = 1.000.000$
- $K = 1$ the minimal number to be able to reach a threshold
- $C_{save} = 15.000.000$ as the runtime test is $[15M - 20M]$
- $C_{load} = 15.000.000$ to better isolate the compute cost, we assume that load and save are symmetric

$$C_{compute} \geq \frac{2 \cdot 15.000.000}{1} + \frac{2 \cdot 15.000.000}{1.000.000}$$
$$C_{compute} \geq 30.000.000 + 30$$

This result is confirmed by the runtime analysis graphs (and underlying benchmarks) in Section 6.5.1.

6.4.2 Checkpoint interval

Another parameter that we can set is the *checkpoint interval (X)* . This is the number of steps the enclave does between each saving. The previous equations don't account for it and it is assumed to be 1, i.e. each step saves the whole payload. Now taking this in the $C_{checkpoint}$ becomes:

$$C_{checkpoint}(X) = \frac{N}{X} \cdot C_{save} \ + \ C_{compute} \cdot N \ + \ C_{load} \cdot K \ + \ C_{compute} \cdot K \cdot \lceil \frac{X}{2} \rceil \quad (6.7)$$

What changed is that the X factor is below the first term, cutting the threshold drastically and a new term appeard, it is the estimated cost or reloading. This estimation is the number of reload needs to recompute, in average, half of the checkpoint interval. This is because the fault can happen anywhere in the range $[0 - X]$ uniformly. Thus giving an estimated of loss of $\lceil \frac{X}{2} \rceil$ per crash. The optimal interval balances these two terms. Taking the derivative of $C_{checkpoint}(X)$ with respect to X and setting it to zero yields:

$$\frac{dC}{dX} = -\frac{N \cdot C_{save}}{X^2} + \frac{C_{compute} \cdot K}{2} = 0$$

Isolating the term in X^2 gives:

$$\frac{N \cdot C_{save}}{X^2} = \frac{C_{compute} \cdot K}{2}$$

Cross-multiplying and solving for X^2:

$$X^2 = \frac{2N \cdot C_{save}}{C_{compute} \cdot K}$$

Taking the square root of both sides gives the closed-form optimal checkpoint interval:

$$X^* = \lceil \sqrt{\frac{2N \cdot C_{save}}{C_{compute} \cdot K}} \rceil \quad (6.8)$$

This result is structurally equivalent to the classical Young/Daly optimal checkpoint interval formula [33], adapted here to a discrete fault-count model K rather than a continuous failures rate. Equation 6.8 shows that X^* grows with the square root of the save cost C_{save} and the total workload N, while it shrinks as the fault count K or the compute cost $C_{compute}$ increases, reflecting the intuition that more frequent or more costly faults justify more frequent checkpointing. We can also note that, compared to the other equations, the number of runs done has a great impact on the resulting optimum.

The resulting value for our implementation is:

$$\begin{aligned} X^*(C_{compute}) &= \lceil \sqrt{\frac{2N \cdot C_{save}}{K \cdot C_{compute}}} \rceil \\ &= \lceil \sqrt{\frac{2 \cdot (1.000.000) \cdot (15.000.000)}{1 \cdot C_{compute}}} \rceil \\ &= \lceil \sqrt{\frac{3 \times 10^{13}}{C_{compute}}} \rceil \end{aligned}$$

Showing that as the computing cost rises, the interval can be smaller in order to for the costs to be break even with the gains. Now, we can try to check that for our implementation with $C_{compute} = 200.000$

$$\begin{aligned} X^*(C_{compute}) &= \lceil \sqrt{\frac{3 \times 10^{13}}{200.000}} \rceil \\ &\approx 12.248 \end{aligned}$$

Thus, until the interval gets to 12 thousand, the ratio will improve, this effect is more visible when the computation cost is closer to the save/load cost or when the numbers of runs get smaller. The figures 6.9, 6.10 and 6.11 are a runtime representation of this effect.

6.5 Runtime Analysis

Now that the theory is established, we will focus on the runtime benchmarks and their resulting graphs. In order to avoid outliers and CPU inconsistency, all the tests have been made a 1000 times, their results have been divided by the same amount. The following subparts are constitued of three graphs, these graphs are all representing the same output data but the cost of the save and load (C_{save}, C_{load}). Our current implementation saves and loads costs are ranging from $15.000.000$ and $20.000.000$ (6.5.2), this is the value given to the first two of each graph. Third one has a set cost of $50.000.000$ for both. The choice has been made to set them to equal values as C_{load} holds less weight for our purpose of long-running tasks (6.3), avoiding unnecessary confusion.

Moreover, the compute cost measured in our implementation is contained within $[60k - 200k]$. It is a simple addition, variable switch and an increment and doesn't represent a real life specific application. The main application being very theoretical, we compare different values in order to avoid making false assumptions of the implementation of a specific task in the enclave. We will discuss the relevance of these posssible values of the three different costs in the Section 7.

6.5.1 Computation threshold

These graphs have been made using the `EAPP_BREAK_EVEN_TESTING` macro along with the `EAPP_BREAK_EVEN_CSV_OUTPUT`.

First, for the fixed $K = 1$. The Figure 6.1 and is the *low expectation* runtime, the ratio is met at around $30m$ as shown in the example equation. Figure 6.2 is the *high expectation* runtime, the ratio is met at around $40m$. To be convinced of this, just replace the parameters of the equation 6.6:

$$C_{compute} \geq \frac{2 \cdot 20.000.000}{1} + \frac{2 \cdot 20.000.000}{1.000.000}$$
$$C_{compute} \geq 40.000.000 + 40$$

As for the last one, the Figure 6.3 can be verified if we just replace the parameters:

$$C_{compute} \geq \frac{2 \cdot 50.000.000}{1} + \frac{2 \cdot 50.000.000}{1.000.000}$$
$$C_{compute} \geq 100.000.000 + 100$$

Secondly for the overview of several possible K, the overall curvature of the function has the same, logarithmic shape. The root of the ratio ($y = 1$) gets shrinked conversely to the number of faults (K). For $K = 0$ the threshold is $\approxeq \infty$, for $K = 1$, it is R (computed with the parameters using 6.6), $K = 2$ is $R/2$,

Finally, for $K = X$ the threshold is $\frac{R}{x}$. This could lead us to think that the usefulness of rewinding is not growing along with the number of faults. This is only if we don't look at the scaling of the functions. Taking the edge cases of the Figure 6.5, for $k = 1$ vs $K = 9$ we can see that when the ratio is only halfway met for the minimal case, it is already close to 3 for the maximum one.

After that, we measure the impact of reducing the period of saving the checkpoint. As shown in the equations, adding some space between checkpoints saves a lot of time, however, making it too big gives negative results as the recovery cost rises along the period. An edge case can be useful to represent that. This is, the period is the same as the numbers of runs, making the rewind effectively not active. This would be equal to a naive enclave.

Overall, the results confirm that the rewind enclave's performance is a balancing game between the computation costs, the final number iterations needed and the number of expected faults. Under well-chosen parameters, and assuming at least one fault, the rewind mechanism appears to be relevant.

Fixed K

First, we fix K to 1 as if it is below that, the threshold cannot be reached and if higher, the threshold will simply be met sooner. Making this value the most interesting one to analyze. These k-

fixed graphs also highlight the possible gap between the average, the maximum and the minimum recorded over the 1000 test runs. This contrasts to the theoretical bound by showing that even if the average is indeed meeting the threshold, an actual run might not be worth the excess computation. On the other hand, we now know that the model behaves as intended and is, most likely, reliable when conditions are met.

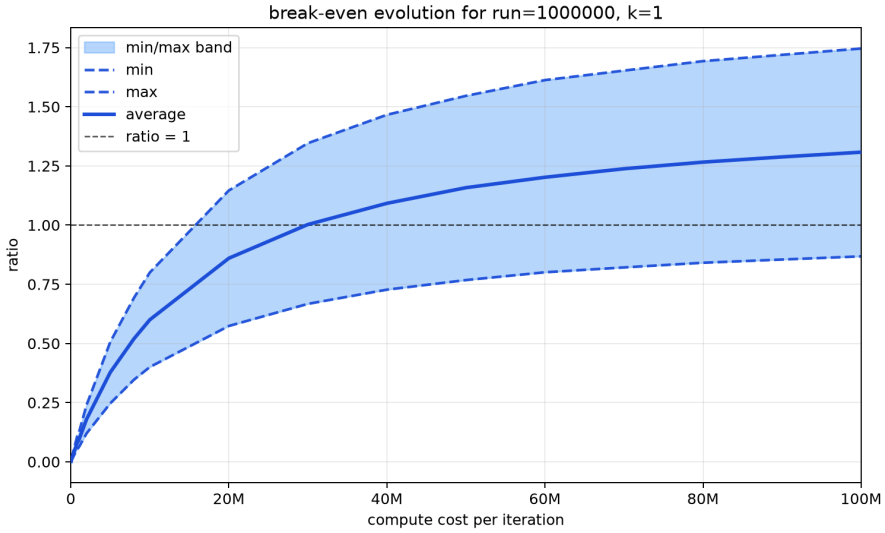

Figure 6.1: Threshold for 1 error over $1m$ runs assuming a 15M cost for load and save

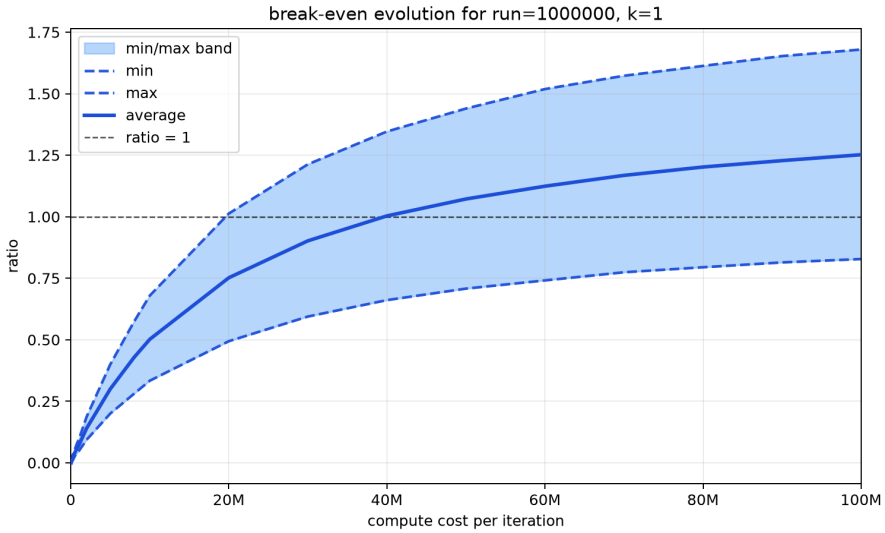

Figure 6.2: Threshold for 1 error over $1m$ runs assuming a $20m$ load/save cost

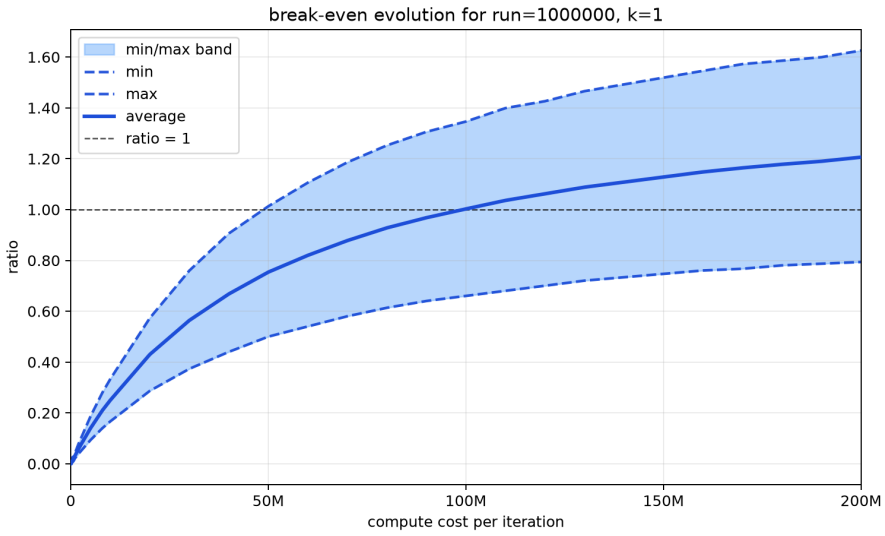

Figure 6.3: Threshold for 1 error over $1m$ runs assuming a $50m$ load/save cost

K = $[0-9]$

Next we show the result, without the maximal and minimal bounds for the first 9 errors. These ones highlight the plateau effect even when working with possibly more faults. As we explained above, the plateau appears because as we increase $C_{compute}$ to balance the other ones, the two sides equal out and both C_{save} and C_{load} become insignificant. Our intuition can also help us understand that effect as both $C_{checkpoint}$ and $C_{no_checkpoint}$ both use $C_{compute}$ Nevertheless, as K grows, the first slope gets steeper, resulting in a **logarithmic shaped function**. We can explain that with the effective cost of the save being shrinked by more faults, in both intuition and equation 6.6. The balance between the two effects gives some kind of saturation slopes.

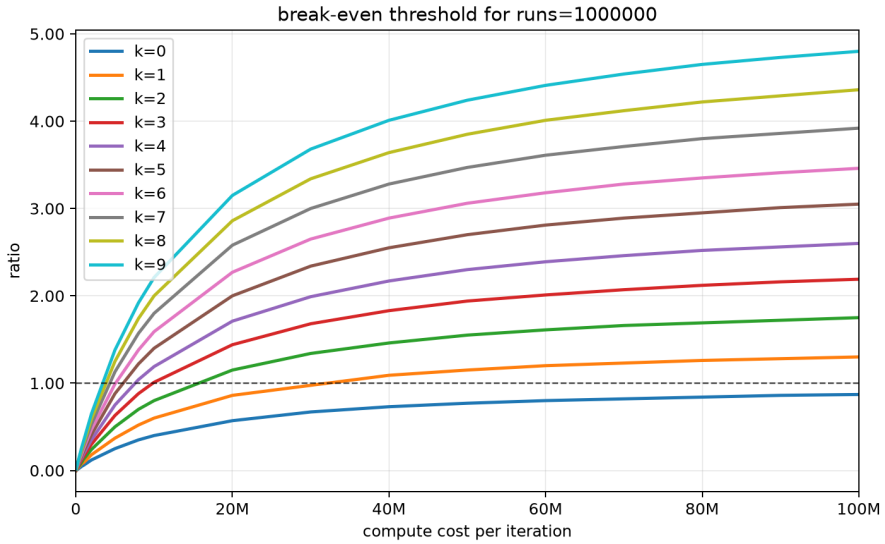

Figure 6.4: Threshold ratio for K=$[0-9]$, over a $1m$ runs enclave assuming a $15m$ load/save cost

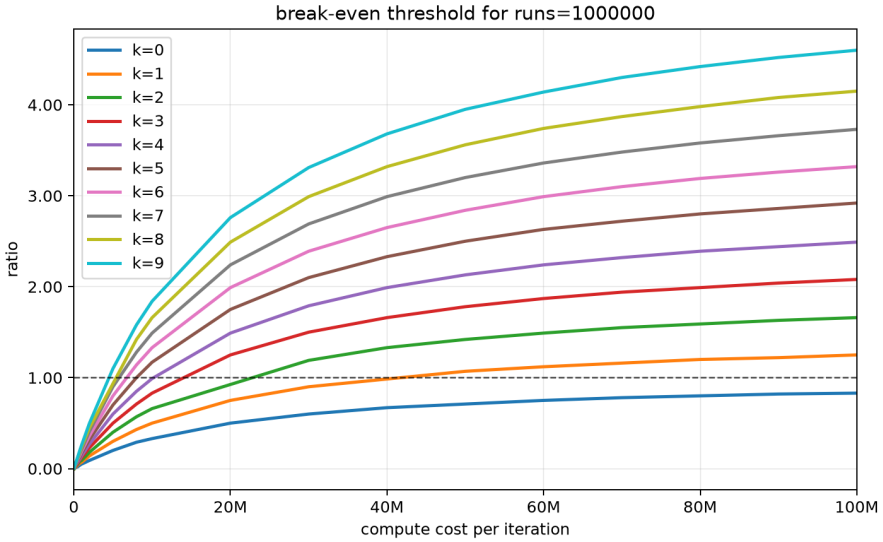

Figure 6.5: Threshold ratio for K=$[0-9]$, over a $1m$ runs enclave assuming a $20m$ load/save cost

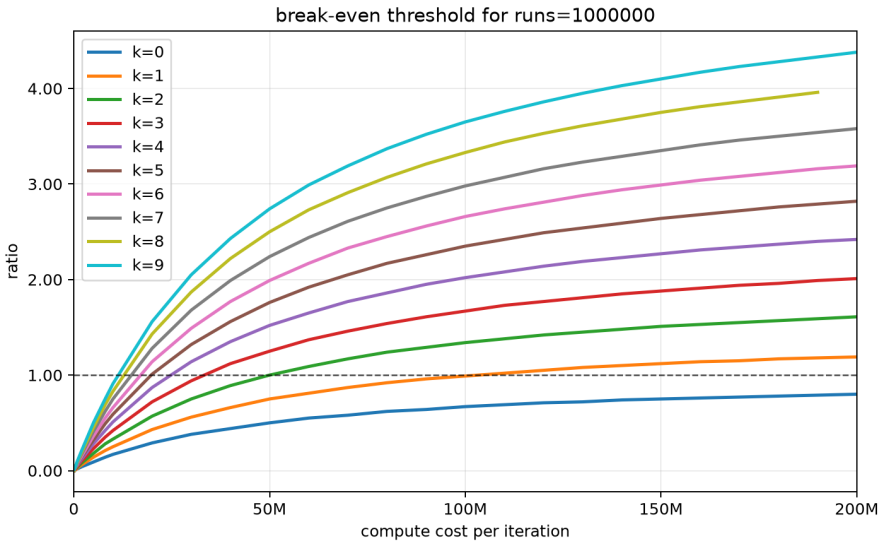

Figure 6.6: Threshold ratio for K=$[0-9]$, over a $1m$ runs enclave assuming a $50m$ load/save cost

Checkpoint interval

We can see that making a save every 2 steps already yield much greater results as they cut the cost needed for the threshold by 2. However, the results rise until $X \leq 12248$, after that, the recovery cost starts balancing out the gains made by avoiding the saved computation. As predicted, the cost of having a wider interval is growing along with the chances of faults occurring.

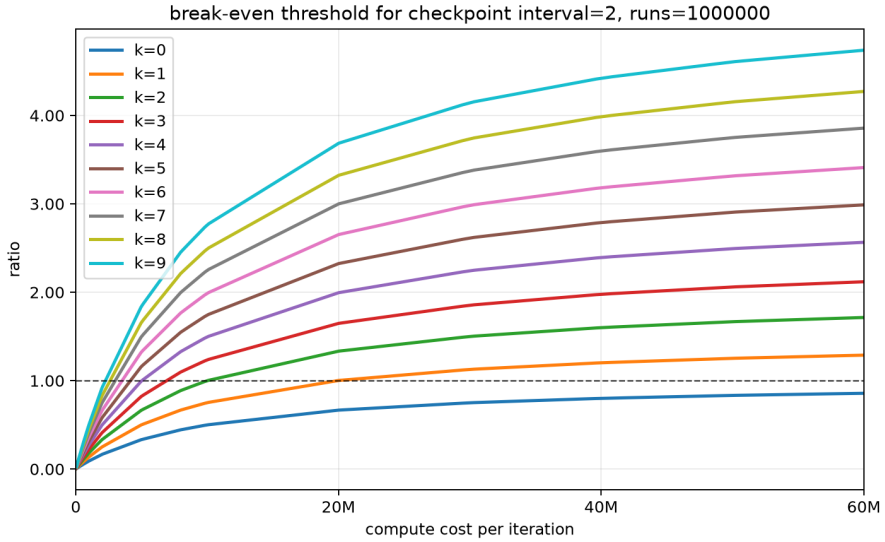

Figure 6.7: Checkpoint interval = 2 threshold ratio for k=$[0-9]$, over $1m$ runs assuming a $20m$ load/save cost

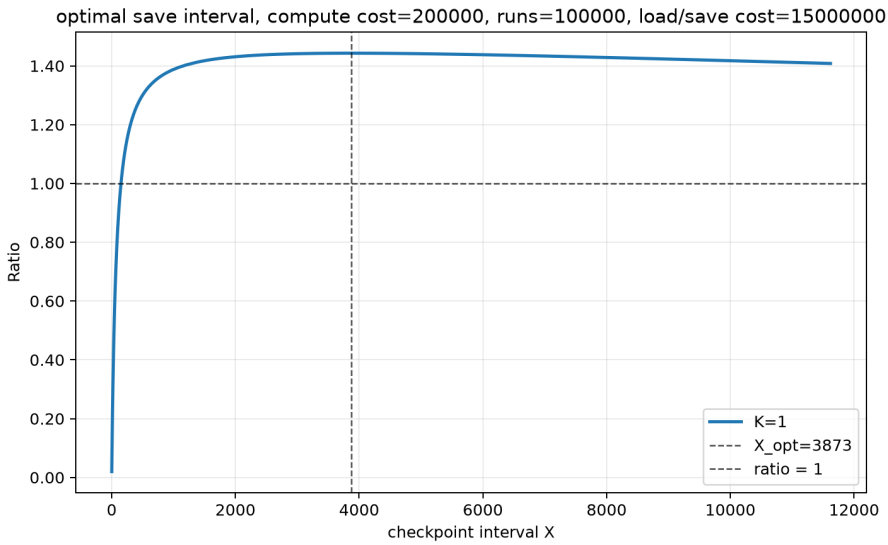

Figure 6.8: Optimal checkpoint interval for $100k$ runs and **1 error**, k=1, $15m$ load/save cost and $200k$ compute cost

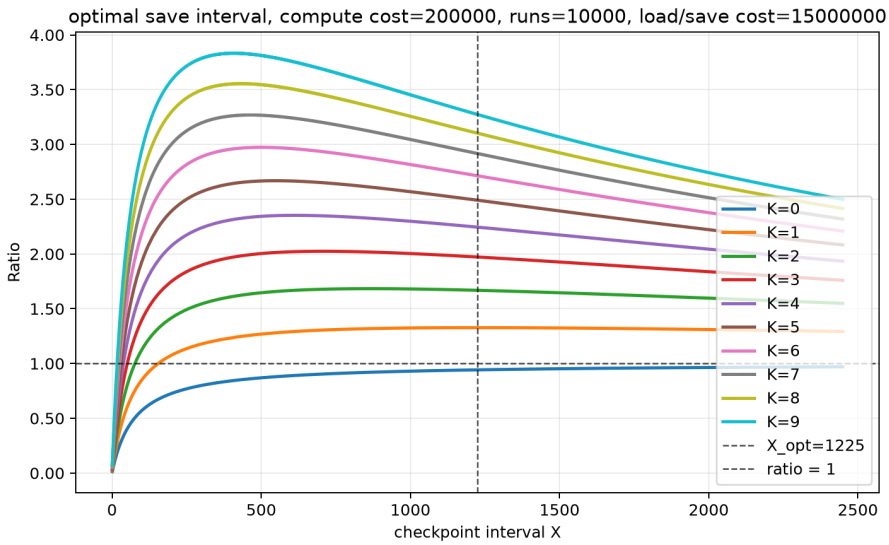

Figure 6.9: Optimal checkpoint interval for $10k$ runs and **1 error**, k=$[0-9]$, $15m$ load/save cost and $200k$ compute cost

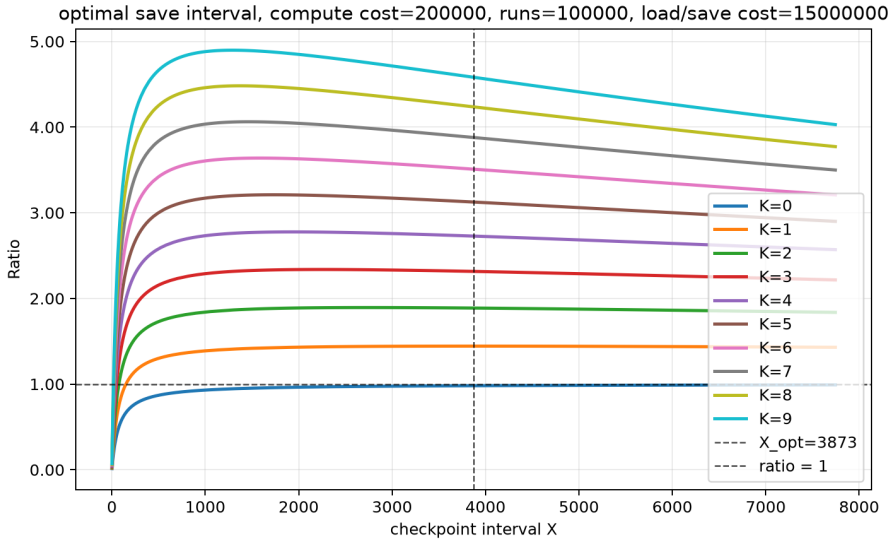

Figure 6.10: Optimal checkpoint interval for $100k$ runs and **1 error**, k=$[0-9]$, $15m$ load/save cost and $200k$ compute cost

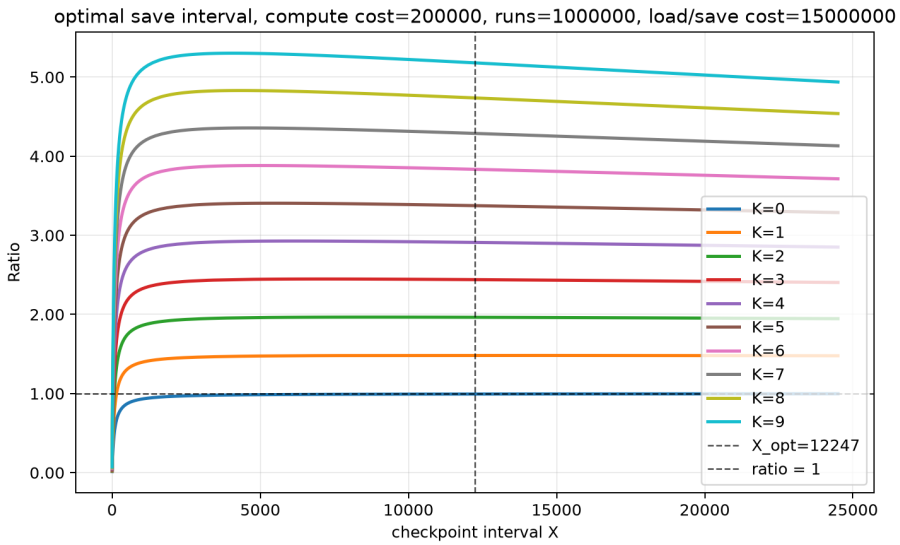

Figure 6.11: Optimal checkpoint interval for $1m$ runs and **1 fault**, k=$[0-9]$, $15m$ load/save cost and $200k$ compute cost

6.5.2 Cycle cost per operation

This part is a recap of each possible main operation of the enclave during an iteration. The values below are averaged over 1000 runs, they include the material derivation as *cached* from the sealing and unsealing and the data being sealed can be found in the Section 5.3.2. We used the `EAPP_CYCLE_BREAKDOWN_TESTING` macro to render these tables.

However, even using a smoothing method to avoid outliers, QEMU is not as reliable as FireSim [22] in terms of cycle usage. Having this in mind, we could not run FireSim because of the long setup it needs. In fact, it has a whole documentation over at: Chipyart documentation. In order to use it, we would need to:

- setup a FireSim Manager instance on Amazon EC2
- deploy it to Amazon AWS FPGAs using FireSim
- install the Chipyard repo onto it
- use it as an external library in Chipyard

The following results must then be interpreted while keeping this limitation in mind.

Component	Cycles
Key derivation	21,873
Seal preparation	9,366
AES-CTR encryption	9,698,615
CBC-MAC tag	9,939,972
Blob copy out	16,089
Total seal cost	19,664,042

Table 6.1: Seal cycle cost breakdown

Component	Cycles
Blob copy in	8,367
CBC-MAC tag recomputation	9,992,689
Tag compare	1,697
AES-CTR decryption	9,702,006
Blob copy out	13,409
Total unseal cost	19,718,168

Table 6.2: Unseal cycle cost breakdown

Operation	Cycles
Computation	275,100

Table 6.3: compute costs

6.5.3 Data cost

The benchmarked plaintext checkpoint has size 8208 bytes, and the measured costs are $19.664.042$ cycles for sealing and $19.718.168$ cycles for unsealing. Using the plaintext size as the denominator gives the per-byte averages:

$$C_{\text{1\_byte\_seal}} \approx \lceil \frac{19.664.042}{8208} \rceil \approx 2396 \tag{6.9}$$

$$C_{\text{1\_byte\_unseal}} \approx \lceil \frac{19.718.168}{8208} \rceil \approx 2402 \tag{6.10}$$

$$C_{\text{1\_byte\_roundtrip}} \approx 4798 \tag{6.11}$$

Time cost

We chose not to analyze the time metrics of the results. Emulating the processor through QEMU is, as stated before, not the most reliable technique. In addition to this weakness, the time can be derived directly from the cycle cost using the frequency of the processor used. In this case, on the RISC-V architecture. This does not add, according to us, any valuable results but, rather, changes the unit of measurement by doing a simple calculation.

6.6 Cybersecurity

Now that the computation and data analysis is done, we will address the cybersecurity assumptions, requirements and objectives.

6.6.1 Security objectives

As stated in 5.2, there are four security objectives that will be looked upon. We will discuss why they hold and what are the limitations. The claims can be checked using the corresponding macros in (Section 6).

Confidentiality

Confidentiality means that the checkpoint is, at any time, opaque from the host. An attacker can, indeed, not read the saved blob because of the `aes_encrypt_ctr()` function provided by Keystone. In our implementation, the rewind state to save is read from the stack, placed into a

plain text checkpoint storage and then sealed using the encryption key derived from Keystone data sealing key (Section `get_sealing_key()` 4.6) and the iv (the checkpoint sequence). After that, a tag is done over the ciphertext using the authentication key to block anyone from tampering without notice. The resulting sealed checkpoint blob is then sent to the host along with the iv and the tag.

Integrity

We now have to consider the possible tampering and side-attacks, the former is that the attackers try to mess with the blob bytes directly, which is not plausible under the scheme *over a fixed-length message*. Another attack is to obtain information using side-channels information. This can be time to compute the tag or reload the enclave by destroying it at every save edge call and XOR-ing the resulting initial ciphertexts. The first one is addressed using a simple, constant time comparison function that always passes over the whole sequence instead of a memcmp() that stops as soon a one byte is wrong. using memcmp(), or any non-constant time comparison function would lead to a time of authentication leak in our scenario. The second one is, in our case, directly addressed by the inherent deterministic mechanism of the enclave, the initial state, will always be the same as the one being compiled (whatever a and b are). Moreover, the checkpoint sequence also follows this rule, XOR-ing of a same ciphertext with different IVs is then simply not possible.

Atomicity

Atomicity states that we cannot have an undefined saved state. Based on the two last objectives, a maliciously modified state is not something to be scared of. However, could an attacker stop the enclave at the perfect time in order to create a chimera state. The answer is no. In fact, the only real ability, to this regard, of the attacker is to cut the power of the enclave. The only outcome of this scenario is to also stop the host and the SM, effectively stopping the whole system. The protection of a whole system is not in the scope of this thesis. Nevertheless, the state, checkpoint of this state and sealed checkpoint are being kept separated for this purpose. Whenever one changes, it doesn't always affect the others.

Another idea would be to compromise the PMP hardware itself, this scenario is not in the scope of this thesis as it is assumed to be sound in runtime to malicious programs. Even only destroying the entry would result in a call to the SM, and a move of the enclave from this hardware space.

Unclonability

We consider a device to be cloned if we can replicate the exact behavior given a sealed state. This is only possible for a program that has the **exact** same hash as the EAPP and is ruled by the SM. Otherwise, the keys won't correspond and the attacker will not be able to read or to use the sealed data. In addition to that, there is no point in time where the plaintext is exposed. We can thus safely rely on Keystone data sealing to make the sealed checkpoint unusable for someone that is not the enclave EAPP.

Chapter 7

Discussion and Future work

7.1 Discussion

The results show that rewind is not a universal replacement for a normal execution path, but it becomes attractive as soon as the enclave performs enough useful work between faults to amortize the cost of saving and restoring state. In practice, the relevant question is not whether checkpointing is expensive in absolute terms, but whether the saved execution time is larger than the overhead introduced by sealing, loading, and recovery. That is why the break-even threshold and the checkpoint interval are the two most important quantities in this thesis. Moreover, a fault could cause a long-time shutdown in practice for long-running jobs. Recovering from that, even in a non-optimal setup, could be useful in time concerns.

The threshold analysis confirms that the cost of checkpointing is dominated by the fixed overhead of saving the enclave state. When the compute cost is small, the mechanism is difficult to justify because the enclave spends most of its time protecting work rather than doing work. As the compute cost increases, the same overhead becomes easier to absorb, and rewind starts to make sense for longer-running or more repetitive tasks. This means the mechanism is most useful for workloads that run long enough to tolerate occasional recovery cost, such as batch-style processing, repeated iterations, or tasks that may have to survive a long interruption.

The checkpoint interval analysis refines that conclusion. Saving too often reduces the amount of work lost after a fault, but it also increases the number of expensive seal operations. Saving too rarely lowers checkpoint overhead, but it increases the amount of recomputation after a failure. The optimal interval therefore depends on the relative cost of computation, checkpointing, and fault recovery, and on the expected number of faults over the lifetime of the enclave. In other words, rewind is only practical if the application can choose an interval that matches its own cost profile rather than relying on a fixed policy for every workload. Another point about the saving ratio is for mixed-criticality systems. The period could be mapped to a criticality level so that most important task gets reloaded quicker while the others could still recover. Thus, all benefiting from the gains of rewinding while mitigating its costs.

From a security point of view, the model is useful because it contains the threat outside the enclave while keeping the recovery state protected. The host remains unreliable, but the check-

pointed state can still be restored safely as long as the sealing and integrity assumptions hold. This gives a clear separation of responsibilities: the untrusted world can fail, delay, or discard the enclave, but it should not be able to read or modify the protected checkpoint without detection. The mechanism is therefore acceptable as a resilience layer only if the enclave boundary, the sealing key, and the state-restore path remain trustworthy.

The crypto cost also matters in the discussion. The current sealing approach is correct for the implementation, but it contributes directly to the runtime overhead and therefore to the break-even point. Any lighter checkpoint protection scheme would improve the usefulness of rewind by lowering the fixed cost of each save and making shorter checkpoint intervals more affordable. This is why the cryptographic choice is not just a security detail: it is part of the performance model.

Overall, the thesis supports a conditional answer. Rewind is useful when the enclave does enough work between faults, when the checkpoint interval is chosen carefully, and when the security boundary can be trusted even though the host cannot. It is less convincing for tiny workloads, for very rare faults, or for applications that cannot afford the extra checkpoint cost. That makes the mechanism promising, but only in the class of long-running protected tasks where recovery cost is smaller than the work it saves.

7.2 Future Work

The following directions extend the current rewind design without changing its basic goal. They focus on making the checkpoint policy more adaptable, the protection layer lighter, and the recovery path more realistic.

7.2.1 Mixed-criticality

The current implementation treats every enclave run as if it had the same level of urgency and the same recovery policy. A natural follow-up would be to support mixed-criticality workloads, where the enclave can distinguish between essential work that must always be preserved and optional work that can be recomputed or discarded more aggressively.

This could make the checkpoint policy more flexible. A high-criticality task could save state more often and use a stricter recovery path, while a lower-criticality task could accept larger intervals or even cheaper checkpoint protection. In that setting, the checkpoint interval would no longer be a single global parameter, but a policy that depends on the value of the task being protected.

Such a design would also make the discussion on usefulness more realistic. Real applications rarely consist of one homogeneous loop, and a mixed-criticality model would let the rewind mechanism adapt to different phases of the same application instead of forcing one compromise for the whole execution.

Nevertheless, the results on checkpoint intervals could be used to implement a higher level criticality-model, that operates at the enclave level. Making each enclave at a given level and then mapping this to a period, the highest priority getting the lowest period. Note that this would need some fine tuning and system design in order to fit the hardware capabilities and task deadlines.

7.2.2 Modern lightweight cryptoscheme

The current sealing layer has been chosen because of its supposed soundness (relying on Keystone data sealing), but it is not necessarily the best choice if the goal is to minimize recovery overhead.

A useful future direction would be to replace it with a more lightweight cryptographic construction that still provides confidentiality and integrity while reducing the cost of every save and load operation.

This matters because the cryptographic cost appears directly in the checkpointing model. Any reduction in sealing overhead would lower the break-even threshold and make shorter checkpoint intervals less expensive. It would also improve the practicality of rewind for workloads with smaller compute cost, which are currently the least favorable case for the mechanism.

Another possible improvement would be to separate the cryptographic policy from the checkpoint logic more clearly using hardware enforced cryptography for example or built-in more specialised schemes. That could make it easier to compare several schemes under the same benchmark framework and to measure whether the gain is mostly coming from a better algorithm, a smaller state, or a better implementation choice.

7.2.3 Fault model complexity

The thesis currently assumes a relatively simple fault model, which is enough to show the basic behavior of rewind but not enough to describe every realistic failure pattern. A stronger follow-up would be to study burst faults, correlated faults, and repeated faults on the same execution window, since those cases can change the usefulness of checkpointing quite dramatically.

This could also let the model capture the difference between isolated failures and adversarial or unstable environments. If the faults are no longer independent, then the expected recomputation cost changes, the optimal checkpoint interval changes, and the threshold analysis becomes more difficult but also more representative of real deployments.

From an implementation point of view, a richer fault model could also allow the enclave to react differently depending on the failure history. For example, the system could decide to shorten the checkpoint interval after repeated faults or to switch to a degraded recovery mode when the fault rate becomes too high.

A strong fault model could allow deeper researches and more targeted results to analyze.

7.2.4 Enclave complexity on backup

The current backup path is intentionally simple: it restores the enclave state and resumes execution without introducing a richer failure-management policy. A further step could be to study how the complexity of the backup enclave affects both security and performance.

This is important because the backup path is part of the trusted computing base. The more logic it contains, the more careful one must be about correctness, memory usage, and attack surface. At the same time, a more capable backup could support better policy decisions, richer diagnostics, or more fine-grained restoration rules.

It would also be useful to measure how much extra checkpoint metadata is needed as the backup logic grows. If recovery requires more state, then the cost of saving and validating that state may erase part of the benefit of rewind. Studying that tradeoff would clarify how far the design can scale (up or down) before the backup mechanism becomes too complex to remain attractive.

7.2.5 Fault detection and correction

For now, the fault detection is solely guided by the fault model. It is purely experimental and no real fault checking is being done. A possible direction is to make a real-life fault detecting routine.

This would add some computations at each step for both the enclaves that use or not the rewind scheme, considering that no one wants an enclave to run while being subject to errors. The work could be done over the whole enclave or just the checkpointed structure using different methods to see and analyze the results.

After that, instead of completely restarting the enclave, there could be a fall-back mechanism that tries and correct the error. This would allow a more consistent execution, in a bitflip for example, but it could also leave the enclave in an uncertain state, thus breaking the atomicity objective. A trade-of between optimization and cybersecurity. Making both of these could also help in differentiating a fault from an error and the possible recoverability of the enclave.

7.2.6 Continuous and asynchronous systems

The last direction that we will discuss is to extend the enclave's work to continuous systems. Such a system could have sensors that give information at not synchronized times, making the enclave work less naive than iterating over a work to be done. There could be an undefined number of steps to be done, making the analysis results different. This would also address the possible entries of the enclave instead of using an unreliable host. For example, specific safe hardware addresses could be given but this could outsource the leak and informations. This could also possibly enable leaving the enclave in a non-coherent state or even help to ease side-channel attacks causing crashes or malformed data input.

Chapter 8

Conclusion

This thesis investigated how to provide secure checkpointing and rewind for applications running inside Keystone enclaves under intermittent and adversarial conditions. We focused on the question: how can an enclave safely persist and restore its internal state through untrusted storage and power loss, without leaking secrets or accepting corrupted state. This problem is interesting because many trusted execution environments, including Keystone, provide strong isolation during execution but offer little structured support for recovery once execution is disrupted, especially when the only persistent medium is controlled by a potentially malicious host. In intermittent and attack-prone settings, this gap limits the practicality of trusted execution environments for long-running or critical services that must tolerate crashes, power cuts, and host tampering while maintaining security guarantees.

Our approach extends the Keystone enclave runtime with a minimal, enclave-centric mechanism that seals checkpointed state to the enclave using measurement-bound keys, authenticates checkpoints to detect tampering, and enforces an atomic recovery boundary on restart. Within the threat model defined in this work, the host, its storage, and communication channels are considered adversarial, while enclave memory and sealing keys remain trusted. We explicitly accept rollback and cloning attacks as out of scope, and instead target confidentiality, integrity, authenticity, atomicity, and unclonability of checkpointed state. The resulting design ensures that corrupted or forged checkpoints are rejected, that partially written blobs cannot cause undefined enclave states, and that only an enclave with the correct measurement can unwrap its own recovery data.

From the implementation and experiments, we learned several key lessons. First, keeping the recovery logic small and explicit inside the enclave runtime is crucial, because even modest complexity in the error and restart paths quickly becomes difficult to reason about, especially under intermittent power and adversarial interference. Second, security and liveliness are tightly coupled in this setting, since sealing and authenticating every checkpoint increases computational cost and recovery latency, so any practical design must balance the frequency and granularity of checkpoints against the performance overhead and the risk of losing progress. Third, we observed that host cooperation is not a reliable abstraction for correctness, so the system must treat host-provided checkpoints purely as untrusted blobs and rely exclusively on enclave-side verification before using them. This cybersecurity aspect, along with the computation cost, took a lot of thinking, model design, implementation choices and tests setup in order to have a valid final shape. We were able to learn a lot about the trade-offs, the limitations and the overall designing skills that come along managing both aspects requires. Finally, the experiments confirmed that it is feasible

to integrate secure rewind into Keystone with moderate overhead, provided that applications adopt a disciplined notion of safe points and explicit state capture.

As a reader, the main takeaway should be that secure checkpointing and rewind in trusted execution environments is both necessary and subtle. It requires coordinated design of threat models, recovery boundaries, and runtime mechanisms, rather than simply encrypting state and assuming safety. You should learn that enclave-centric sealing and authentication can provide strong protection against host tampering without extending trust to the host, that atomic recovery semantics are essential to avoid inconsistent states after crashes, and that intermittent or adversarial environments force us to think of enclaves not just as isolated compute islands but as systems that must safely span multiple executions through untrusted media. The work also illustrates how even a relatively lightweight change in the enclave runtime can significantly alter the security and reliability properties of Keystone-based applications.

If starting again, there are several aspects we could have approached differently. We would first invest more effort in a formalization of the recovery boundary and comprehension of the platform workflow, clearly specifying which parts of enclave state must be captured, which invariants must hold at each checkpoint, and how these invariants are re-established after rewind. This would help guide both implementation and testing, and make it easier to reason about corner cases. Second, we would design and integrate benchmarks and adversarial test harnesses earlier in the process, to evaluate performance and failure behavior under more realistic workloads and attack scenarios. Third point is to explore a more modular interface between applications and the checkpointing mechanism, so that different applications can select appropriate policies, such as checkpoint frequency and state granularity, while reusing the core secure sealing and recovery primitives. Another one is to start writing down all the implementation details and ideas sooner. This would have certainly helped in having a clearer idea and helped in seeing the possible gaps that needed to be addressed. Finally, we would examine the interaction with other Keystone features and multi-enclave scenarios sooner, to understand how compartmentalization and cross-enclave dependencies affect recovery. Overall, the implementation part is the first big step that has been done in order to conquer the fear of starting this thesis but as much as it has helped starting, we were too focused on this task for a considerable period of time. Instead, we should have taken a step back sooner to be able to read, write or design the future steps.

The approach developed in this thesis is particularly useful in scenarios where services must maintain long-lived, security-critical state inside enclaves while facing unreliable power or untrusted hosts. Examples include intermittent devices that rely on Keystone for secure computation but frequently lose power, edge services that process sensitive data near users while depending on cloud hosts that cannot be fully trusted, and privacy-preserving analytics systems that run in trusted execution environments but need to survive crashes and restarts without leaking internal state or accepting corrupted histories. We would recommend using our solution for services that have the following characteristics. They store secrets or integrity-critical metadata inside the enclave that must not be exposed or silently corrupted by the host. They can define clear, application-level safe points where state is consistent and can be safely checkpointed. They tolerate the extra overhead of sealing and authentication in exchange for stronger recovery guarantees, using a more costly computation function and-or running for a long time with system that a prone to errors. They operate in environments where power loss, crashes, or deliberate disruption by the host are realistic

threats.

Under these conditions, secure checkpointing and rewind becomes a practical tool for extending Keystone's isolation guarantees across time, not just across memory. While this work does not address rollback prevention, it demonstrates that carefully designed enclave-centric recovery mechanisms can significantly increase resilience in the face of intermittent execution and adversarial hosts, and it lays a foundation for more advanced recovery and coordination schemes in future work.

Bibliography

[1] Hadi Esmaeilzadeh et al. “Dark Silicon and the End of Multicore Scaling”. In: *IEEE Micro* 32.3 (2012), pp. 122–134. DOI: 10.1109/MM.2012.17.

[2] John von Neumann. “Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components”. In: *Automata Studies*. Ed. by C. E. Shannon and J. McCarthy. Princeton, NJ: Princeton University Press, 1956, pp. 43–98.

[3] A. Avizienis. “The N-Version Approach to Fault-Tolerant Software”. In: *IEEE Transactions on Software Engineering* SE-11.12 (1985), pp. 1491–1501. DOI: 10.1109/TSE.1985.231893.

[4] Federico Reghenzani, Zhishan Guo, and William Fornaciari. “Software Fault Tolerance in Real-Time Systems: Identifying the Future Research Questions”. In: *ACM Comput. Surv.* 55.14s (July 2023). ISSN: 0360-0300. DOI: 10.1145/3589950. URL: https://doi.org/10.1145/3589950.

[5] Inseok Hwang et al. “A Survey of Fault Detection, Isolation, and Reconfiguration Methods”. In: *IEEE Transactions on Control Systems Technology* 18.3 (2010), pp. 636–653. DOI: 10.1109/TCST.2009.2026285.

[6] Sepideh Safari et al. “A Survey of Fault-Tolerance Techniques for Embedded Systems From the Perspective of Power, Energy, and Thermal Issues”. In: *IEEE Access* PP (Jan. 2022). DOI: 10.1109/ACCESS.2022.3144217.

[7] IVÁN SERRANO HERNÁNDEZ. “SOFTWARE-LEVEL REDUNDANCY AND DIVERSITY FOR OBJECT DETECTION IN SAFETY-CRITICAL EMBEDDED SYSTEMS”. MA thesis. Universitat Politècnica de Catalunya (UPC) - BarcelonaTech, 2024. URL: https://upcommons.upc.edu/entities/publication/0ded2aa4-21b1-4e30-bb35-f5206e1e5010.

[8] B. Nagajayanthi. “Decades of Internet of Things Towards Twenty-first Century: A Research-Based Introspective”. In: *Wireless Personal Communications* 123 (2022), pp. 3661–3697. DOI: 10.1007/s11277-021-09308-z. URL: https://doi.org/10.1007/s11277-021-09308-z.

[9] Shohreh Hosseinzadeh et al. “Diversification and obfuscation techniques for software security: A systematic literature review”. In: *Information and Software Technology* 104 (2018), pp. 72–93. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2018.07.007. URL: https://www.sciencedirect.com/science/article/pii/S0950584918301484.

[10] Pietro Chiavassa et al. “Secure Intermittent Computing with ARM TrustZone on the Cortex-M”. In: *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. 2024, pp. 160–168. DOI: 10.1109/EuroSPW61312.2024.00022.

[11] Antonio Carzaniga, Andrea Mattavelli, and Mauro Pezzè. “Measuring Software Redundancy”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 1. 2015, pp. 156–166. DOI: 10.1109/ICSE.2015.37.

[12] Dayeol Lee et al. “Keystone: An Open Framework for Architecting Trusted Execution Environments”. In: *Proceedings of the Fifteenth European Conference on Computer Systems*. EuroSys’20. 2020.

[13] John Shalf. “The future of computing beyond Moore’s Law”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 378.2166 (2020), p. 20190061. DOI: 10.1098/rsta.2019.0061. eprint: https://royalsocietypublishing.org/doi/pdf/10.1098/rsta.2019.0061. URL: https://royalsocietypublishing.org/doi/abs/10.1098/rsta.2019.0061.

[14] Institute of Electrical and Electronics Engineers (IEEE) is a global community for technologists, helping to shape the systems and standards of tomorrow. URL: https://www.ieee.org/.

[15] Magne Vollan Aarset. “How to Identify a Bathtub Hazard Rate”. In: *IEEE Transactions on Reliability* R-36.1 (1987), pp. 106–108. DOI: 10.1109/TR.1987.5222310.

[16] Jens Lienig and Hans Bruemmer. *Fundamentals of Electronic Systems Design*. Vol. 1. Springer International Publishing, Apr. 2017, pp. xiii + 241. ISBN: 978-3-319-55840-0. DOI: 10.1007/978-3-319-55840-0. URL: https://link.springer.com/book/10.1007/978-3-319-55840-0.

[17] Andreas Löfwenmark and Simin Nadjm-Tehrani. “Fault and timing analysis in critical multi-core systems: A survey with an avionics perspective”. In: *Journal of Systems Architecture* 87 (2018), pp. 1–11. ISSN: 1383-7621. DOI: https://doi.org/10.1016/j.sysarc.2018.04.001. URL: https://www.sciencedirect.com/science/article/pii/S1383762117304903.

[18] Sepideh Safari et al. “On the Scheduling of Energy-Aware Fault-Tolerant Mixed-Criticality Multicore Systems with Service Guarantee Exploration”. In: *IEEE Transactions on Parallel and Distributed Systems* 30.10 (2019), pp. 2338–2354. DOI: 10.1109/TPDS.2019.2907846.

[19] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. “Trusted Execution Environment: What It is, and What It is Not”. In: *2015 IEEE Trustcom/BigDataSE/ISPA*. Vol. 1. 2015, pp. 57–64. DOI: 10.1109/Trustcom.2015.357.

[20] Gianluca Scopelliti et al. “End-to-End Security for Distributed Event-driven Enclave Applications on Heterogeneous TEEs”. In: *ACM Trans. Priv. Secur.* 26.3 (June 2023). ISSN: 2471-2566. DOI: 10.1145/3592607. URL: https://doi.org/10.1145/3592607.

[21] Merve Gülmez et al. “Rewind & Discard: Improving Software Resilience using Isolated Domains”. In: *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 2023, pp. 402–416. DOI: 10.1109/DSN58367.2023.00046.

[22] Keystone Enclave Project. *Keystone: Open-Source RISC-V Trusted Execution Environment Framework*. https://keystone-enclave.org. Accessed from October 2025 trough August 2026. 2021.

[23] Keystone Enclave Project. *Keystone Documentation*. https://docs.keystone-enclave.org/en/latest/index.html. Accessed 2026-07-26. 2026.

[24] Andrew Waterman and Krste Asanović. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. Tech. rep. Document Version 20191214-draft. RISC-V Foundation, Dec. 2019.

[25] Markus Switalla. “A Review of the Keystone Trusted Execution Framework”. In: 2023. URL: https://api.semanticscholar.org/CorpusID:259367604.

[26] SiFive Inc. *SiFive Freedom Metal library*. https://sifive.github.io/freedom-metal-docs/devguide/pmps.html. 2019.

[27] Peter Maydell et al. *QEMU Website Licensing*. Accessed July 2026. 2014. URL: https://www.qemu.org/license.html.

[28] 2022. URL: https://github.com/keystone-enclave/keystone-runtime.

[29] Sagar Karandikar. “FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud”. MA thesis. EECS Department, University of California, Berkeley, Dec. 2018. URL: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2018/EECS-2018-154.html.

[30] László Szekeres et al. “SoK: Eternal War in Memory”. In: *2013 IEEE Symposium on Security and Privacy*. 2013, pp. 48–62. DOI: 10.1109/SP.2013.13.

[31] Guy L. Steele, Doug Lea, and Christine H. Flood. “Fast splittable pseudorandom number generators”. In: *SIGPLAN Not.* 49.10 (Oct. 2014), pp. 453–472. ISSN: 0362-1340. DOI: 10.1145/2714064.2660195. URL: https://doi.org/10.1145/2714064.2660195.

[32] Guy L. Steele, Doug Lea, and Christine H. Flood. “Fast splittable pseudorandom number generators”. In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications*. OOPSLA ’14. Portland, Oregon, USA: Association for Computing Machinery, 2014, pp. 453–472. ISBN: 9781450325851. DOI: 10.1145/2660193.2660195. URL: https://doi.org/10.1145/2660193.2660195.

[33] J.T. Daly. “A higher order estimate of the optimum checkpoint interval for restart dumps”. In: *Future Generation Computer Systems* 22.3 (2006), pp. 303–312. ISSN: 0167-739X. DOI: https://doi.org/10.1016/j.future.2004.11.016. URL: https://www.sciencedirect.com/science/article/pii/S0167739X04002213.

Appendix

Rewind source code

These files (and their usage) can be found in the repository: GitHub repository and they are compiled (downloadable) in this pdf.

```
# scripts
make-examples.sh
make-run.sh
run-rewind.sh

# source files
examples/rewind/
  CMakeLists.txt
  eapp/
    checkpoint.c
    checkpoint.h
    common.c
    common.h
    crypto.c
    crypto.h
    eapp_test.c
    eapp_test.h
    fault.c
    fault.h
    rewind.c    # enclave file
  host/
    host_test.cpp
    host_test.hpp
    host.cpp
    host.hpp
```

`eapp/checkpoint`

```
examples/rewind/eapp/checkpoint.c
examples/rewind/eapp/checkpoint.h
```

eapp/eapp_test

examples/rewind/eapp/eapp_test.c
examples/rewind/eapp/eapp_test.h

eapp/crypto

examples/rewind/eapp/crypto.c
examples/rewind/eapp/crypto.h

eapp/common

examples/rewind/eapp/common.c
examples/rewind/eapp/common.h

eapp/fault

examples/rewind/eapp/fault.c
examples/rewind/eapp/fault.h

eapp/rewind.c

examples/rewind/eapp/rewind.c

host/host_test

examples/rewind/host/host_test.cpp
examples/rewind/host/host_test.hpp

host/host

examples/rewind/host/host.cpp
examples/rewind/host/host.hpp

CMakeLists.txt

examples/rewind/CMakeLists.txt

Scripts

make-examples.sh

./make-examples.sh

run-rewind.exp

./run-rewind.exp

make-run.sh

```
./make-run.sh
```

README

```
README.md
```